Database Management Systems (DBMS)

Project Report

Smart Hospital Management System

Student Name(s):	*[to be filled in]*
Roll Number(s):	*[to be filled in]*
Branch & Section:	*[to be filled in]*
Course Instructor:	Priyankar
Academic Session:	2026-2027 (Monsoon Semester)

Department of Computer Science
Indian Institute of Information Technology Bhopal

Contents

General Instructions **ii**

1 Introduction **1**
1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1
1.4 Scope of the Project . . . 1
1.5 Software and Hardware Requirements . . . 2

2 System Requirements and Analysis **2**
2.1 Functional Requirements . . . 2
2.2 Business Rules . . . 2

3 Entity–Relationship (ER) Modeling **3**
3.1 Identification of Entities . . . 3
3.2 Identification of Relationships . . . 4
3.3 ER Diagram . . . 4
3.4 Explanation of the ER Diagram . . . 4

4 ER-to-Relational Mapping **14**
4.1 Mapping Rules . . . 14
4.2 Step-by-Step Conversion . . . 15
4.3 Final Relational Schema . . . 15

5 Functional Dependencies **16**
5.1 Identification of Functional Dependencies . . . 16
5.2 Candidate Keys and Attribute Classification . . . 17
5.3 FD Analysis . . . 18

6 Database Normalization **18**
6.1 Unnormalized Form (UNF) . . . 18
6.2 First Normal Form (1NF) . . . 18
6.3 Second Normal Form (2NF) . . . 19
6.4 Third Normal Form (3NF) . . . 19
6.5 Boyce–Codd Normal Form (BCNF) . . . 19
6.6 Final Normalized Schema . . . 20

7 Relational Integrity and Constraints **21**

8 MySQL Database Implementation **22**
8.1 Database Creation . . . 22
8.2 Table Creation . . . 22
8.3 Data Insertion . . . 24
8.4 Data Modification and Deletion . . . 24
8.5 Sample Database Contents . . . 24
8.6 SQL Query Implementation . . . 25
8.7 Advanced DBMS Features . . . 27

9 Application Development **28**
9.1 System Architecture . . . 28
9.2 Front-End Technologies . . . 28
9.3 Application Modules . . . 28
9.4 Database Connectivity . . . 28
9.5 Application Screenshots . . . 29
9.6 Database Operations through the Application . . . 29

10 Testing and Results **30**
10.1 Test Cases . . . 30
10.2 Query Testing . . . 30
10.3 Constraint Testing . . . 30
10.4 Results and Discussion . . . 30

11 Conclusion and Future Scope **31**
11.1 Conclusion . . . 31
11.2 Limitations . . . 31
11.3 Future Scope . . . 31

References **31**

A Complete MySQL Script **32**

B Source Code **32**

C Additional Screenshots **33**

D Team Contribution **34**

Suggested Evaluation Scheme **34**

General Instructions

The course template sets these instructions. The right-hand column says where this report and its submission meet each one.

	Instruction	In this project
1	The front end may use any language or framework.	React 18 single-page app with a Node.js/Express API (Chapter 9).
2	MySQL is mandatory for creation, storage and queries.	MySQL 9.7.1 (InnoDB) is the only database; every table, constraint, view, trigger and procedure is in MySQL (Chapters 7, 8).
3	Design the database before the application.	ER model (Chapter 3), mapping (Chapter 4), dependencies and normalization (Chapters 5, 6) precede the implementation chapters; schema changes are versioned migrations.
4	Demonstrate the DBMS concepts of the course.	Keys, constraints, joins, subqueries, views, triggers, transactions and locking (Chapters 7, 8).
5	Include diagrams, dependencies, normalization steps, SQL and results.	All outputs shown are from real runs on the test database.
6	Submit the report, source code and a complete MySQL script.	Appendix A (`hospital_db_full.sql`) and Appendix B (repository).
7	Every member must understand the design.	Chapter 3.4 and Chapter 6 explain every design decision in words.

Chapter 1

Introduction

1.1 Project Overview

The **Smart Hospital Management System** is a web application that runs the daily work of a hospital on one MySQL database. It covers the path of a patient from registration to discharge: appointments and visits, triage vital signs, doctors' notes, laboratory orders and results, prescriptions dispensed from a batch-tracked pharmacy, medication given on the ward, bed and emergency allocation, admissions, billing and an audit trail of who did what.

Its users are the hospital's staff, each with a role that decides what they may see and do:

- **Receptionist:** registers patients, books appointments, marks arrivals, raises bills.
- **Nurse:** works the wards they are assigned to: triage, vital signs, the medication administration record (MAR).
- **Doctor:** sees the patients in their care, writes notes, orders tests, prescribes, admits and discharges.
- **Laboratory staff:** process lab orders and record results with optional report files.
- **Pharmacy staff:** dispense prescriptions and receive or adjust stock by batch.
- **Administrator:** manages staff accounts, nurse assignments, and reads the audit log and reports.

1.2 Problem Statement

In many hospitals these records live on paper or in separate systems: a register at the front desk, a ward book, a pharmacy stock ledger and a lab file. This causes problems that a shared database prevents:

- **Inconsistent copies:** a patient's details are written several times and drift apart.
- **Unsafe concurrent use:** two clerks can allocate the same last ICU bed, or two pharmacists dispense the same stock.
- **No integrity checks:** a dose can be recorded for the wrong patient, stock can go negative, a result can be filed without an order.
- **No accountability:** it is hard to tell who changed or viewed a record.

1.3 Objectives

1. Design a normalized relational schema (BCNF where sound) for hospital operations, from an ER model.
2. Enforce the hospital's rules in the database itself: keys, foreign keys, CHECK constraints, triggers and transactions.
3. Keep concurrent operations correct: bed allocation, dispensing and status changes use row locks in a fixed order.
4. Give each role a web front end that reads and writes the database only through a restricted MySQL account.
5. Record every data change and refused access in an append-only audit log.
6. Verify all of this with automated tests that run against a separate test database.

1.4 Scope of the Project

Covered: patient registration (including the 14-digit ABHA number), appointments, the encounter (visit) life-cycle, vital signs, consultations, surgery requests, laboratory orders and results with attachments, prescriptions with structured dosing, a batch-level pharmacy with first-expiry-first-out dispensing, the MAR, wards, beds, admissions and discharges, emergency cases with ambulances and automatic bed and doctor allocation, billing, staff accounts, nurse ward assignments, doctor schedules and daily statistics, and the audit log. **Not covered:** payments through a gateway, insurance claims, live integration with the national ABDM health-record network, and multi-hospital operation (Section 11.2).

1.5 Software and Hardware Requirements

Item	Used in this project
Database	MySQL 9.7.1, InnoDB, isolation level REPEATABLE READ, `utf8mb4_0900_ai_ci`, strict SQL mode
Back end	Node.js 25 with Express 4, `mysql2` (promise API, connection pool), JSON Web Tokens, bcrypt, Multer (uploads), Socket.IO 4
Front end	React 18, React Router 6, Vite 4, Axios, Socket.IO client
Testing	Node's built-in test runner (`node:test`); Puppeteer for browser tests
Tools	Git and GitHub, Visual Studio Code, MySQL command-line client and `mysqldump`, draw.io, LaTeX
Deployment	Docker Compose on an Azure B2als_v2 virtual machine (2 vCPU, 4 GiB RAM), Caddy for HTTPS
Minimum hardware	2 CPU cores, 4 GB RAM and 2 GB of disk for MySQL, the API and the browser client; any modern browser for users

Chapter 2

System Requirements and Analysis

2.1 Functional Requirements

The system must support these operations (the role that performs each is in brackets):

- **Patients:** register, search (name, phone, MRN, ABHA), view and edit a patient; record an age when the date of birth is unknown [Receptionist, Administrator].
- **Appointments and visits:** book an appointment with a doctor's slot, mark arrival, triage, start and finish the visit, or cancel it [Receptionist, Nurse, Doctor].
- **Clinical records:** record vital signs [Nurse]; write consultation notes, order lab tests, prescribe, request surgery [Doctor].
- **Laboratory:** move an order to processing, record the result (auto-flagged against reference ranges), attach the report file [Laboratory].
- **Pharmacy:** dispense a prescription from non-expired batches, earliest expiry first; receive and adjust stock by batch; low-stock and expiry views [Pharmacy].
- **Ward care:** see assigned wards and beds, record each scheduled dose as given, refused or missed [Nurse].
- **Beds, admissions and emergencies:** admit to a free bed, discharge, register emergencies and allocate a bed and doctor [Doctor, Receptionist, Administrator].
- **Billing:** create bills, add line items, record payments [Receptionist, Administrator].
- **Administration:** manage staff accounts and nurse ward assignments; read the audit log; dashboards and reports [Administrator].

2.2 Business Rules

Every rule below is enforced by the schema or by the application code, as stated in the last column.

	Rule	Enforced by
1	Every patient has a unique MRN (the patient id); an ABHA number, when given, is 14 digits and unique.	PK `patients.patient_id`; UNIQUE `uq_patients_abha`; CHECK `chk_patients_abha_format`
2	A date of birth cannot be in the future.	API validation (`POST/PUT /api/patients`, 400)
3	A doctor cannot be booked twice for the same date and time.	UNIQUE (`doctor_id, appointment_date, appointment_time`)
4	A visit moves only ARRIVED → (TRIAGED) → IN_PROGRESS → FINISHED; it can be CANCELLED before it starts or marked ENTERED_IN_ERROR before it finishes; its appointment's status follows it.	State machine in `utils/encounters.js`, in one transaction with the appointment row locked
5	A patient has at most one ACTIVE admission, and a bed holds at most one.	Locking reads (`SELECT ... FOR UPDATE`) in the admission and emergency services; triggers mark the bed OCCUPIED / AVAILABLE

	Rule	Enforced by
6	A doctor's workload counts their ACTIVE admissions and never exceeds their maximum.	Trigger `after_admission_insert` / `after_admission_discharge`; CHECK `doctors_chk_1`
7	Stock can never become negative.	CHECK on `medicines` and `medicine_batches`; trigger `before_medicine_stock_update`
8	A medicine's stock is the sum of its batches; dispensing takes non-expired batches, earliest expiry first.	Same-transaction updates with rows locked in medicine-id order (`dispenseService`)
9	A prescription item is dispensed at most once, and a dispense time implies it was dispensed.	Guard `AND dispensed = FALSE` (409 on a second dispense); CHECK `chk_prescription_items_dispensed`
10	A record filed under a visit, admission, note or batch belongs to the same patient (or medicine).	Composite foreign keys `fk_ctx_*` (15) and API checks (400)
11	A bill's paid amount never exceeds its total; a line's total is quantity × unit price.	CHECK `bills_chk_2`; generated column `bill_items.total_price`
12	An order has at most one result.	UNIQUE `lab_results.order_id`
13	A dose recorded as given has a time; a scheduled dose exists once per order and slot.	CHECK `chk_mar_given`; UNIQUE (`prescription_item_id, scheduled_at_utc`)
14	Vital signs are within physiological ranges and belong to a visit or an admission.	Eleven CHECKs `chk_vitals_*`
15	Discharge is not before admission; a visit does not end before it starts.	CHECK `admissions_chk_1`, `chk_encounters_times`
16	A user account has at most one doctor profile.	UNIQUE `uq_doctors_user_id`
17	Each action is limited to the roles that may perform it; a doctor sees only patients in their care.	`authorize(...)` middleware on every route; care-set check in `utils/patientAccess.js`
18	The audit log can only grow.	Triggers `audit_logs_no_update` and `audit_logs_no_delete`

Chapter 3

Entity–Relationship (ER) Modeling

3.1 Identification of Entities

The model has 30 entity types in six modules. Each becomes one table (Chapter 4); the two M:N and ternary relationships with attributes become the other two tables, and `schema_migrations` records applied schema versions and is not part of the domain. Table 3.1 lists every entity with its identifier; the diagrams show each entity's principal attributes, and Section 4.3 lists all 341 columns. Four entities are **weak** (Section 3.4).

Table 3.1: Entities, their module, table and identifier

Entity	Mod.	Table	Type	Identifier (key / partial key)
PATIENT	1	`patients`	strong	patient_id
APPOINTMENT	1	`appointments`	strong	appointment_id
ENCOUNTER	1	`encounters`	strong	encounter_id
VITAL_SIGN	1	`vital_signs`	strong	vital_id
CONSULTATION	1	`consultations`	strong	consultation_id
PRESCRIPTION	2	`prescriptions`	strong	prescription_id
PRESCRIPTION_ITEM	2	`prescription_items`	strong	item_id
MED_ADMINISTRATION	2	`medication_administrations`	strong	administration_id
SURGERY_REQUEST	2	`surgery_requests`	strong	request_id
MEDICINE	3	`medicines`	strong	medicine_id
MEDICINE_BATCH	3	`medicine_batches`	weak	batch_number
STOCK_MOVEMENT	3	`pharmacy_stock_movements`	strong	movement_id
LAB_TEST	4	`lab_tests`	strong	test_id
LAB_ORDER	4	`lab_orders`	strong	order_id
LAB_RESULT	4	`lab_results`	strong	result_id
LAB_ATTACHMENT	4	`lab_result_attachments`	strong	attachment_id
WARD	5	`wards`	strong	ward_id
BED	5	`beds`	weak	bed_number
ADMISSION	5	`admissions`	strong	admission_id
EMERGENCY_CASE	5	`emergency_cases`	strong	emergency_id
AMBULANCE	5	`ambulances`	strong	ambulance_id
BILL	5	`bills`	strong	bill_id
BILL_ITEM	5	`bill_items`	strong	item_id
USER	6	`users`	strong	user_id

Entity	Mod.	Table	Type	Identifier
DOCTOR	6	`doctors`	strong	doctor_id
DEPARTMENT	6	`departments`	strong	department_id
DOCTOR_SCHEDULE	6	`doctor_schedules`	weak	day_of_week, start_time
DAILY_ANALYTICS	6	`doctor_daily_analytics`	weak	analytics_date
WARD_ASSIGNMENT	6	`nurse_ward_assignments`	strong	assignment_id
AUDIT_LOG	6	`audit_logs`	strong	audit_id

3.2 Identification of Relationships

There are 69 relationship types. 64 are binary 1:N, three are binary 1:1 (BECOMES, YIELDS, IS_DOCTOR) and two carry attributes: FILLED_FROM is M:N (a prescription line can be filled from several batches, and a batch fills many lines) and OCCUPIES is ternary (bed, admission, patient). Together they account for every one of the 72 relationship foreign keys of the schema; a script checks this, and checks each participation and cardinality below against the column's nullability and unique keys. In Table 3.2 the ratio and participation are given in the order the entities are listed (T = total, P = partial); * marks an identifying relationship.

Table 3.2: Relationships, cardinality ratios and participation

Relationship	Entities	Ratio	Part.	Relationship	Entities	Ratio	Part.
BOOKS	PATIENT – APPOINTMENT	1:N	P/T	OF_TEST	LAB_TEST – LAB_ORDER	1:N	P/T
ATTENDS	DOCTOR – APPOINTMENT	1:N	P/T	OFFERS	DEPARTMENT – LAB_TEST	1:N	P/P
VISITS	PATIENT – ENCOUNTER	1:N	P/T	TESTED	PATIENT – LAB_ORDER	1:N	P/T
CONDUCTS	DOCTOR – ENCOUNTER	1:N	P/T	ORDERS	DOCTOR – LAB_ORDER	1:N	P/T
BECOMES	APPOINTMENT – ENCOUNTER	1:1	P/P	LAB_IN	ENCOUNTER – LAB_ORDER	1:N	P/P
OPENS	USER – ENCOUNTER	1:N	P/P	LAB_FROM	CONSULTATION – LAB_ORDER	1:N	P/P
STAY_VISIT	ADMISSION – ENCOUNTER	1:N	P/P	STAY_LAB	ADMISSION – LAB_ORDER	1:N	P/P
MEASURED	PATIENT – VITAL_SIGN	1:N	P/T	YIELDS	LAB_ORDER – LAB_RESULT	1:1	P/T
TAKEN_IN	ENCOUNTER – VITAL_SIGN	1:N	P/P	ANALYSES	USER – LAB_RESULT	1:N	P/P
TAKEN_ON	ADMISSION – VITAL_SIGN	1:N	P/P	ATTACHED	LAB_RESULT – LAB_ATTACHMENT	1:N	P/T
RECORDS	USER – VITAL_SIGN	1:N	P/T	UPLOADS	USER – LAB_ATTACHMENT	1:N	P/T
NOTE_OF	PATIENT – CONSULTATION	1:N	P/T	HAS_BED*	WARD – BED	1:N	P/T
WRITES	DOCTOR – CONSULTATION	1:N	P/T	IN_DEPT	DEPARTMENT – WARD	1:N	P/T
NOTE_IN	ENCOUNTER – CONSULTATION	1:N	P/P	ADMITTED	PATIENT – ADMISSION	1:N	P/T
NOTE_FOR	APPOINTMENT – CONSULTATION	1:N	P/P	ADMITS	DOCTOR – ADMISSION	1:N	P/T
STAY_NOTE	ADMISSION – CONSULTATION	1:N	P/P	ADMITTED_TO	DEPARTMENT – ADMISSION	1:N	P/T
PRESCRIBED	PATIENT – PRESCRIPTION	1:N	P/T	ALLOCATED	BED – ADMISSION	1:N	P/T
PRESCRIBES	DOCTOR – PRESCRIPTION	1:N	P/T	OCCUPIES	BED – ADMISSION – PATIENT	N:M:1	P/T/P
RX_IN	ENCOUNTER – PRESCRIPTION	1:N	P/P	RESULTS_IN	EMERGENCY_CASE – ADMISSION	1:N	P/P
RX_FROM	CONSULTATION – PRESCRIPTION	1:N	P/P	ARRIVES	PATIENT – EMERGENCY_CASE	1:N	P/P
STAY_RX	ADMISSION – PRESCRIPTION	1:N	P/P	CARRIES	AMBULANCE – EMERGENCY_CASE	1:N	P/P
LISTS	PRESCRIPTION – PRESCRIPTION_ITEM	1:N	P/T	ASSIGNED	DOCTOR – EMERGENCY_CASE	1:N	P/P
OF_DRUG	MEDICINE – PRESCRIPTION_ITEM	1:N	P/T	GIVEN_BED	BED – EMERGENCY_CASE	1:N	P/P
FILLED_FROM	PRESCRIPTION_ITEM – MEDICINE_BATCH	M:N	P/P	BILLED	PATIENT – BILL	1:N	P/T
GIVEN_AS	PRESCRIPTION_ITEM – MED_ADMINISTRATION	1:N	P/T	STAY_BILL	ADMISSION – BILL	1:N	P/P
STAY_DOSE	ADMISSION – MED_ADMINISTRATION	1:N	P/P	ITEMISES	BILL – BILL_ITEM	1:N	P/T
GIVEN_BY	USER – MED_ADMINISTRATION	1:N	P/P	IS_DOCTOR	USER – DOCTOR	1:1	P/T
SURGERY_OF	PATIENT – SURGERY_REQUEST	1:N	P/T	WORKS_IN	DEPARTMENT – DOCTOR	1:N	P/T
REQUESTS	DOCTOR – SURGERY_REQUEST	1:N	P/T	HAS_SLOT*	DOCTOR – DOCTOR_SCHEDULE	1:N	P/T
SURGERY_IN	ENCOUNTER – SURGERY_REQUEST	1:N	P/P	HAS_STATS*	DOCTOR – DAILY_ANALYTICS	1:N	P/T
HAS_BATCH*	MEDICINE – MEDICINE_BATCH	1:N	P/T	NURSE_OF	USER – WARD_ASSIGNMENT	1:N	P/T
RECEIVES	USER – MEDICINE_BATCH	1:N	P/P	ASSIGNS	USER – WARD_ASSIGNMENT	1:N	P/P
MOVES	MEDICINE – STOCK_MOVEMENT	1:N	P/T	COVERS	WARD – WARD_ASSIGNMENT	1:N	P/T
FROM_BATCH	MEDICINE_BATCH – STOCK_MOVEMENT	1:N	P/P	LOGS	USER – AUDIT_LOG	1:N	P/P
PERFORMS	USER – STOCK_MOVEMENT	1:N	P/T				

3.3 ER Diagram

The diagrams use Chen notation (legend in Figure 3.1). Figure 3.2 shows the six modules and the 39 relationships between modules. Figures 3.3–3.11 show the modules one page at a time; three modules take two pages. An entity drawn in another module appears as a dashed reference box ("see Module n"), and every relationship between two modules is drawn in both, with the same name and cardinality.

3.4 Explanation of the ER Diagram

Reading the diagrams. A line ends at an entity with the cardinality that entity has in the relationship. For example, BOOKS reads "one PATIENT books N APPOINTMENTs". A double line means every instance of that entity takes part: every appointment is for a patient (`appointments.patient_id` is NOT NULL), while a patient need not have an appointment (single line). Optional links, such as a prescription written outside a visit, are partial on both sides.

Weak entities. An entity is weak here only when the schema identifies it within its owner, that is, its natural identifier is unique only together with the owner's key and the owner is mandatory:

- **BED** (owner WARD, partial key bed_number): bed numbers are unique only within a ward, UNIQUE (`ward_id, bed_number`).
- **MEDICINE_BATCH** (owner MEDICINE, partial key batch_number): a supplier's batch number is unique only for one medicine, UNIQUE (`medicine_id, batch_number`).
- **DOCTOR_SCHEDULE** (owner DOCTOR, partial key day_of_week, start_time): a slot exists only as a slot of one doctor, UNIQUE (`doctor_id, day_of_week, start_time`), deleted with the doctor (ON

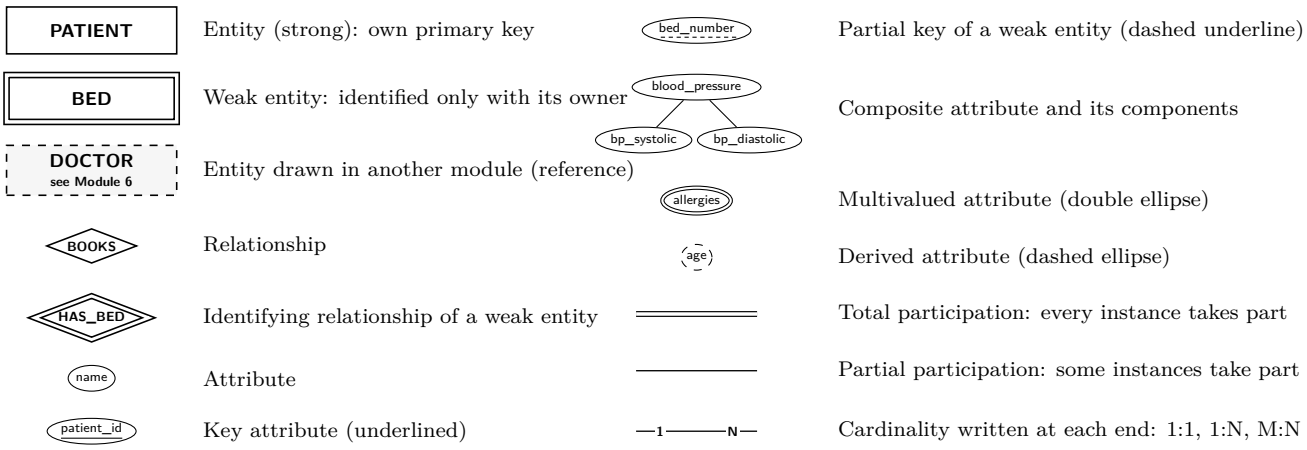


Figure 3.1: Legend of the Chen notation used in all ER diagrams

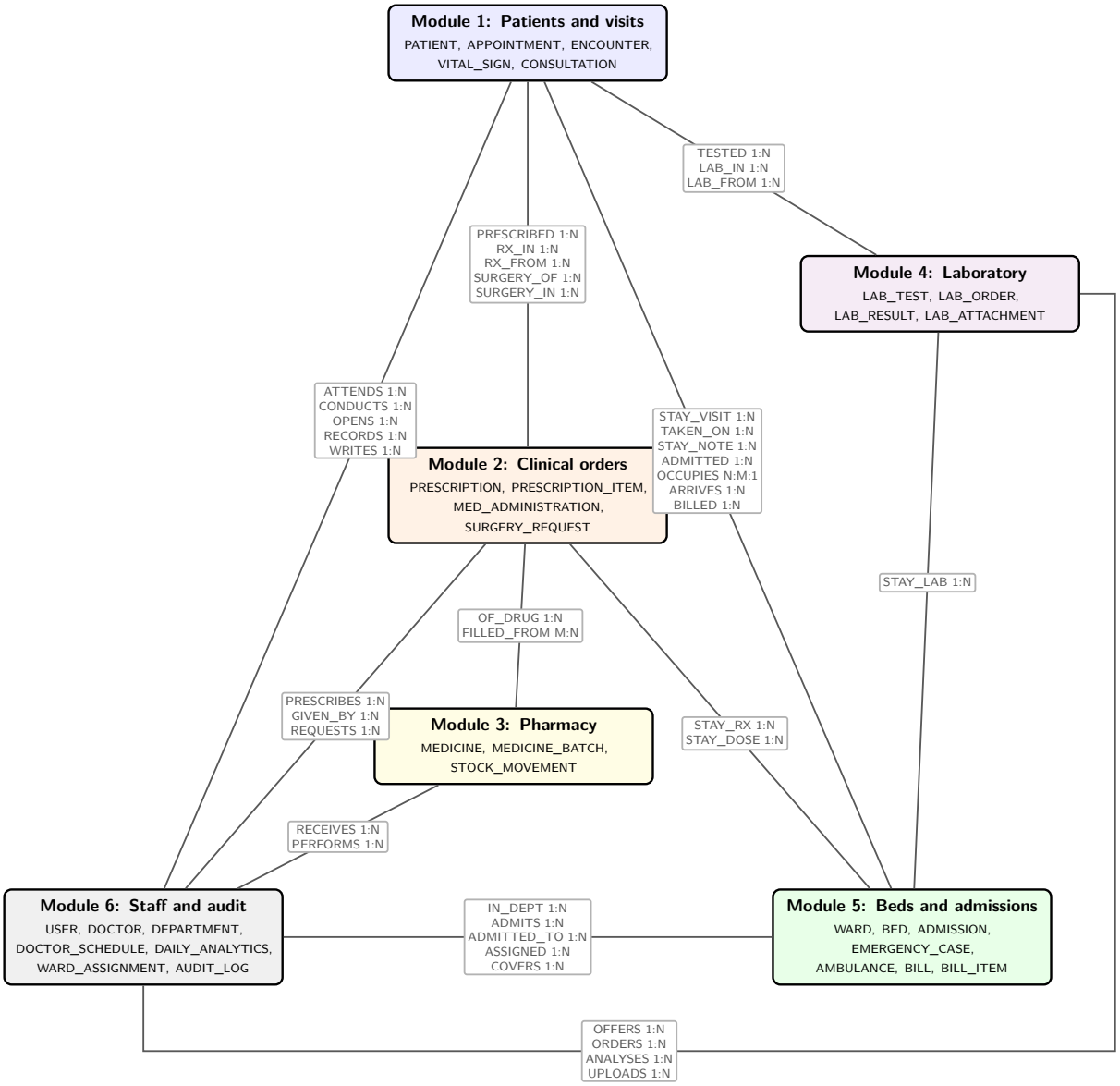


Figure 3.2: Overview: the six modules and every relationship that crosses between two modules (name and cardinality ratio)

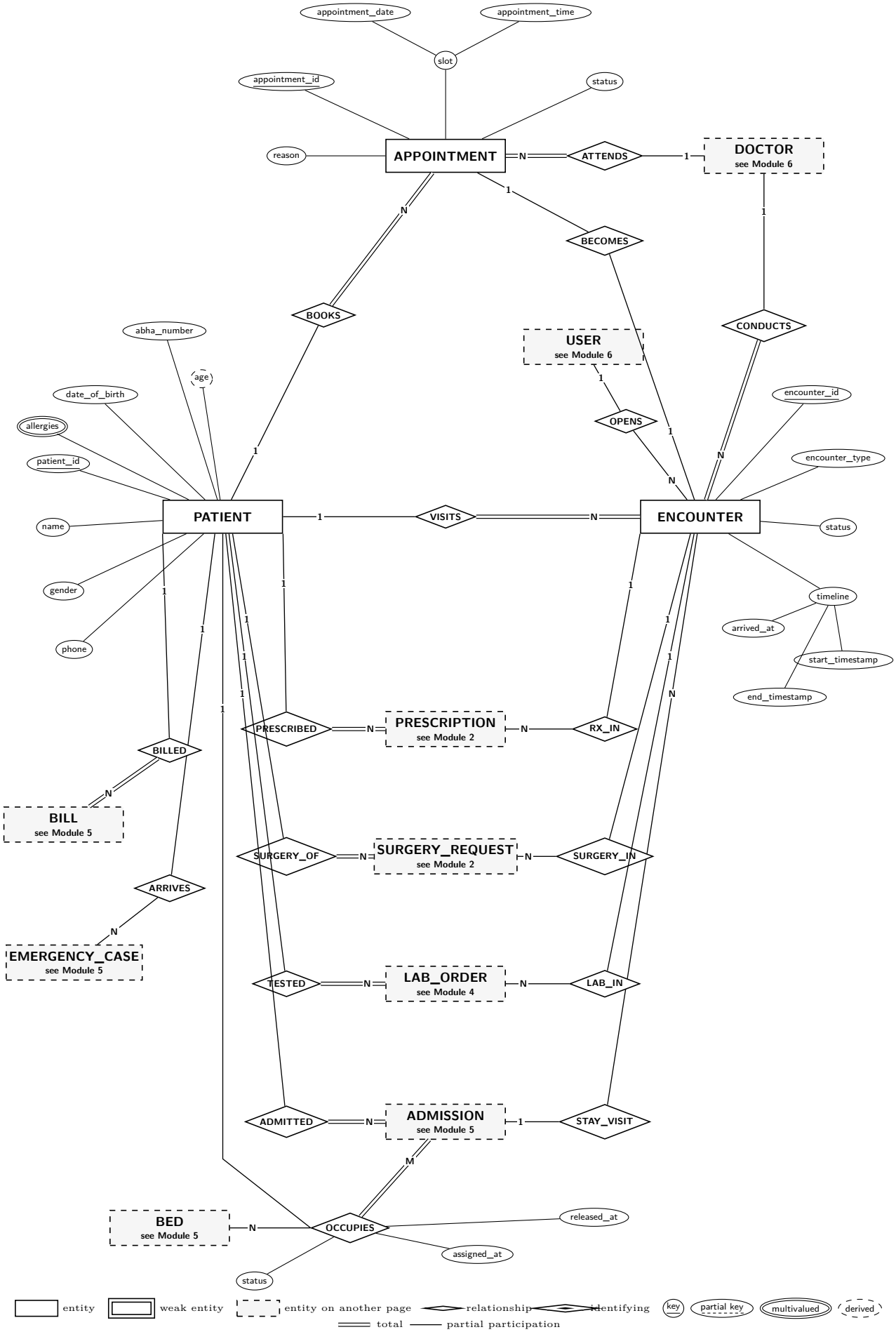


Figure 3.3: Module 1, Patients and visits (page 1 of 2): patients, appointments and encounters

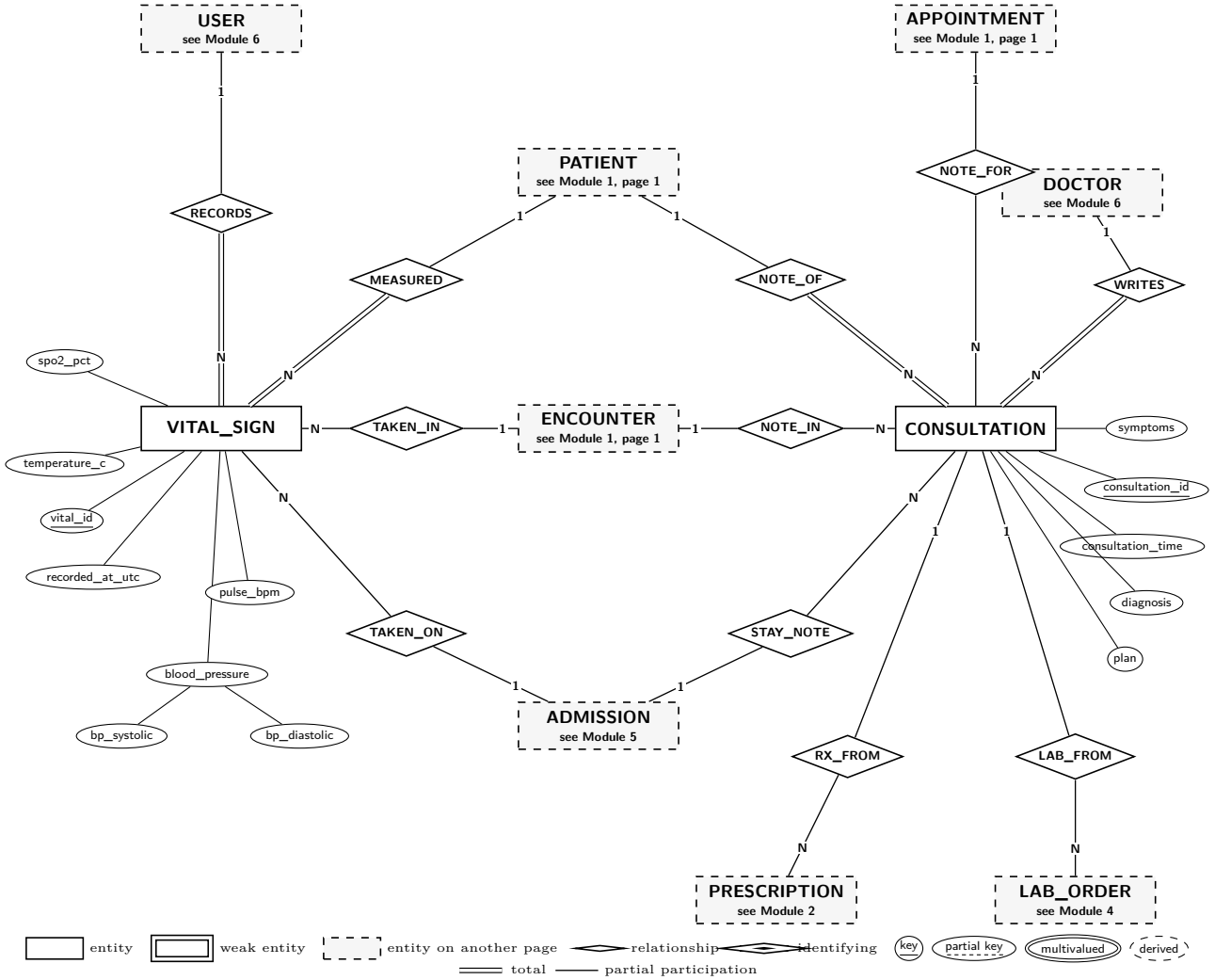


Figure 3.4: Module 1, Patients and visits (page 2 of 2): vital signs and consultations

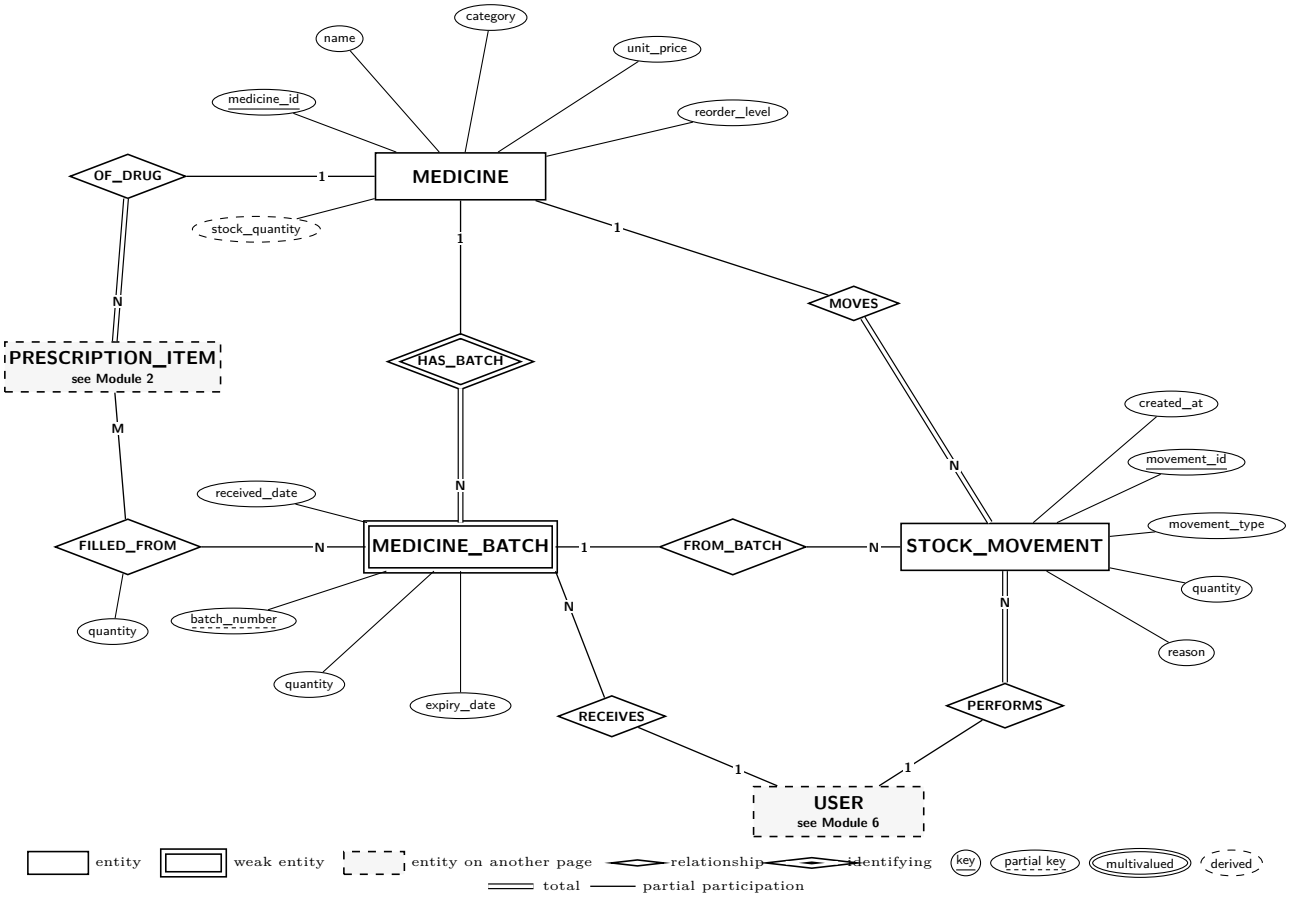


Figure 3.5: Module 3, Pharmacy: medicines, batches and stock movements

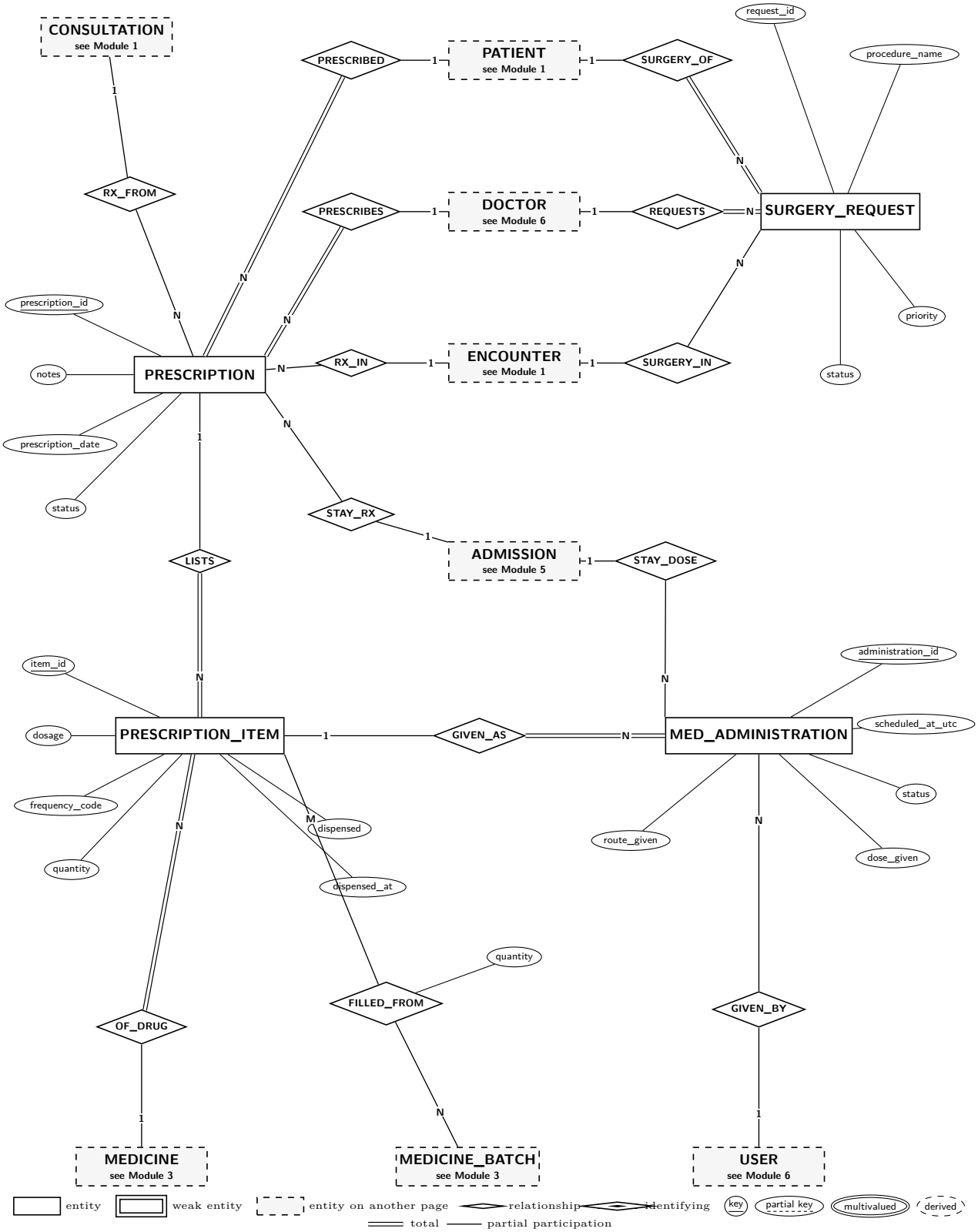


Figure 3.6: Module 2, Clinical orders: prescriptions, prescription lines, medication administration and surgery requests

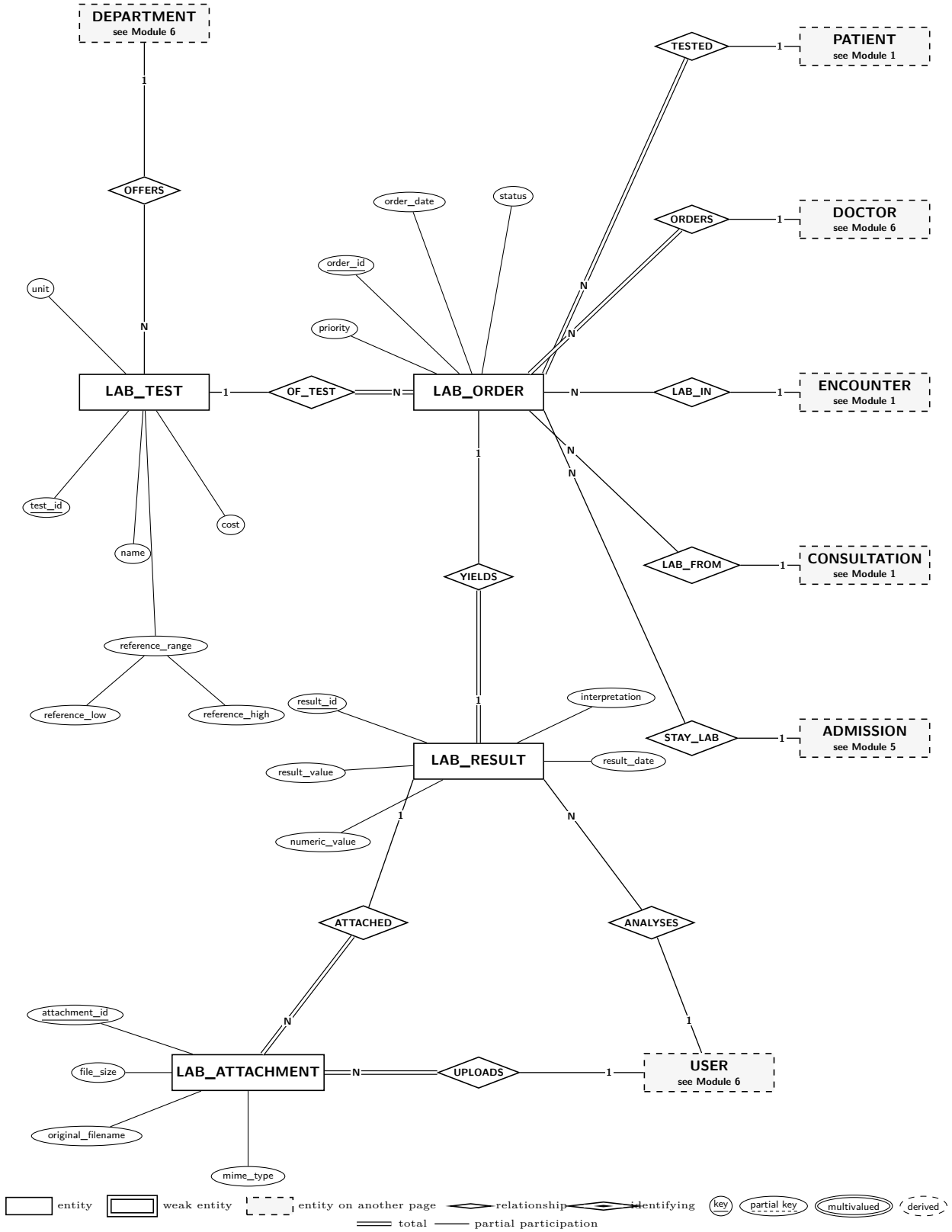


Figure 3.7: Module 4, Laboratory: tests, orders, results and report files

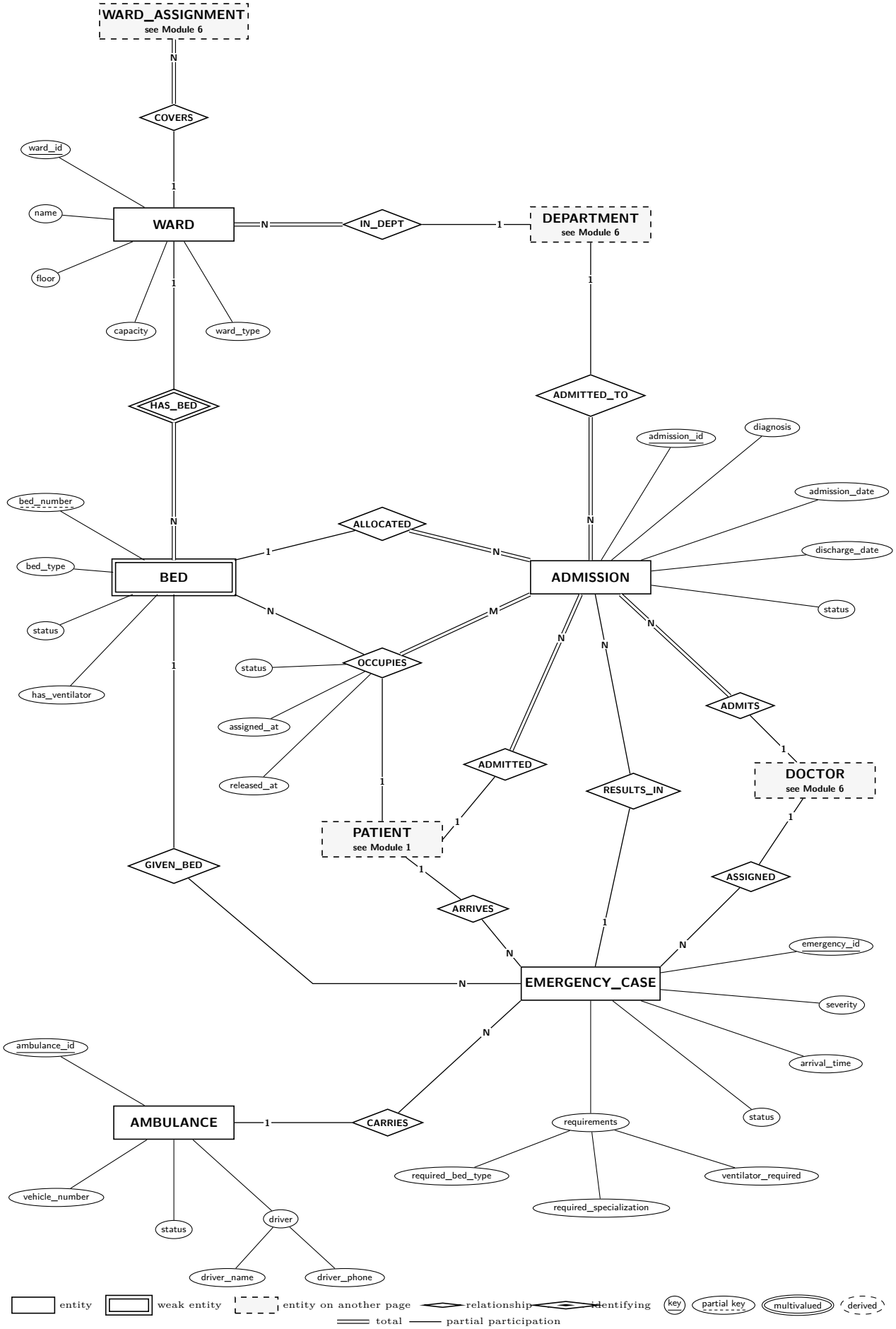


Figure 3.8: Module 5, Beds and admissions (page 1 of 2): wards, beds, admissions, emergencies and ambulances

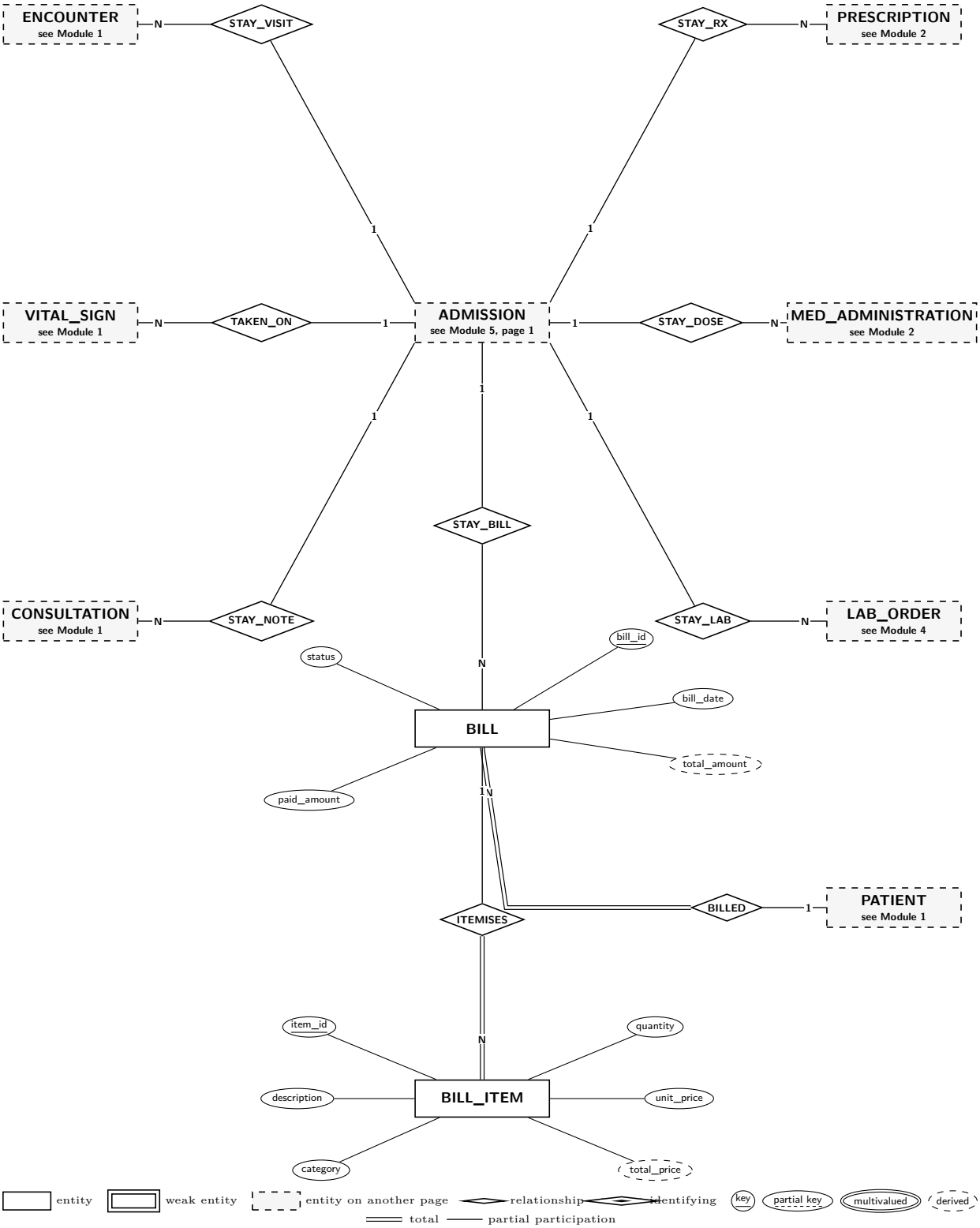


Figure 3.9: Module 5, Beds and admissions (page 2 of 2): what an admission covers, and billing

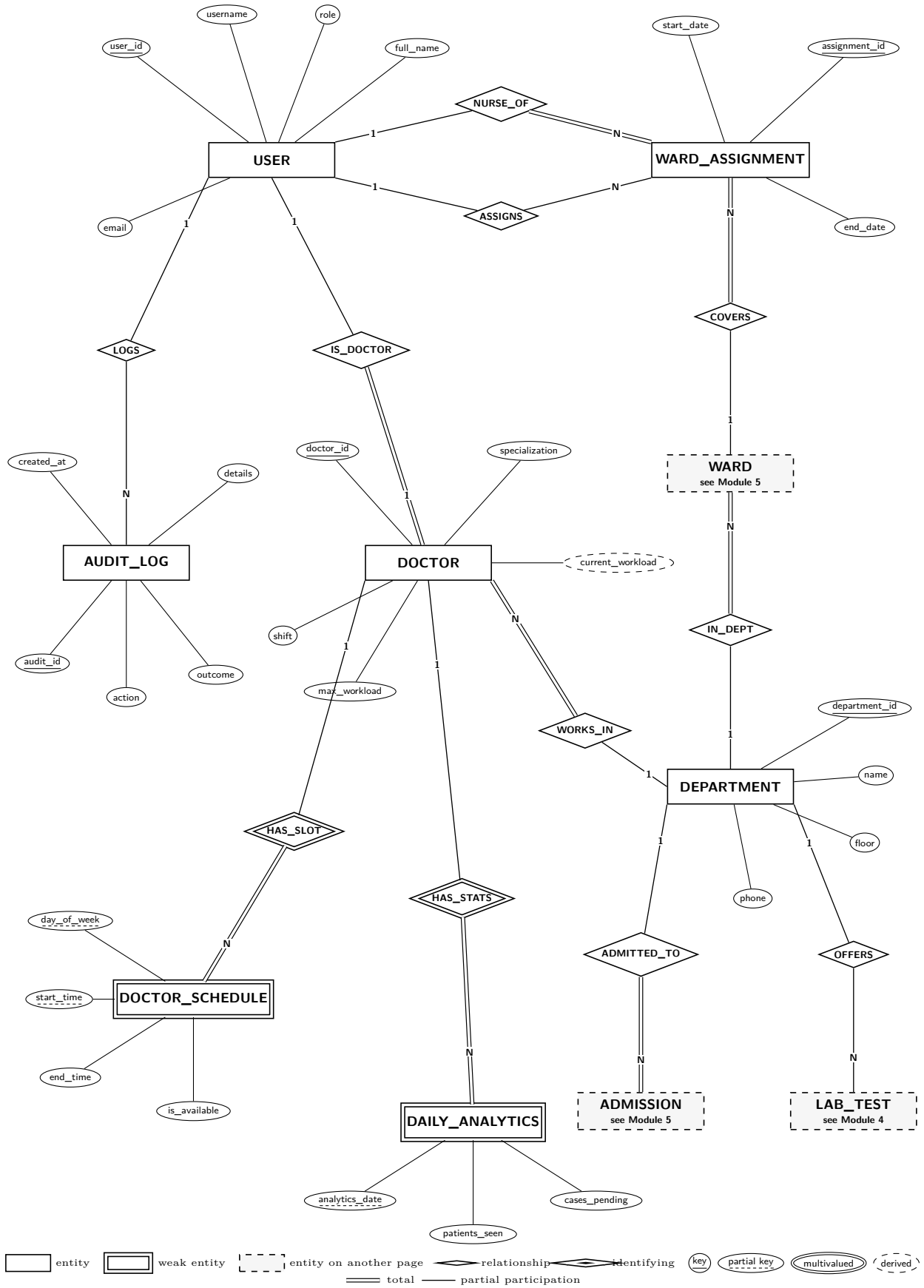


Figure 3.10: Module 6, Staff and audit (page 1 of 2): accounts, doctors, departments, assignments, schedules and the audit log

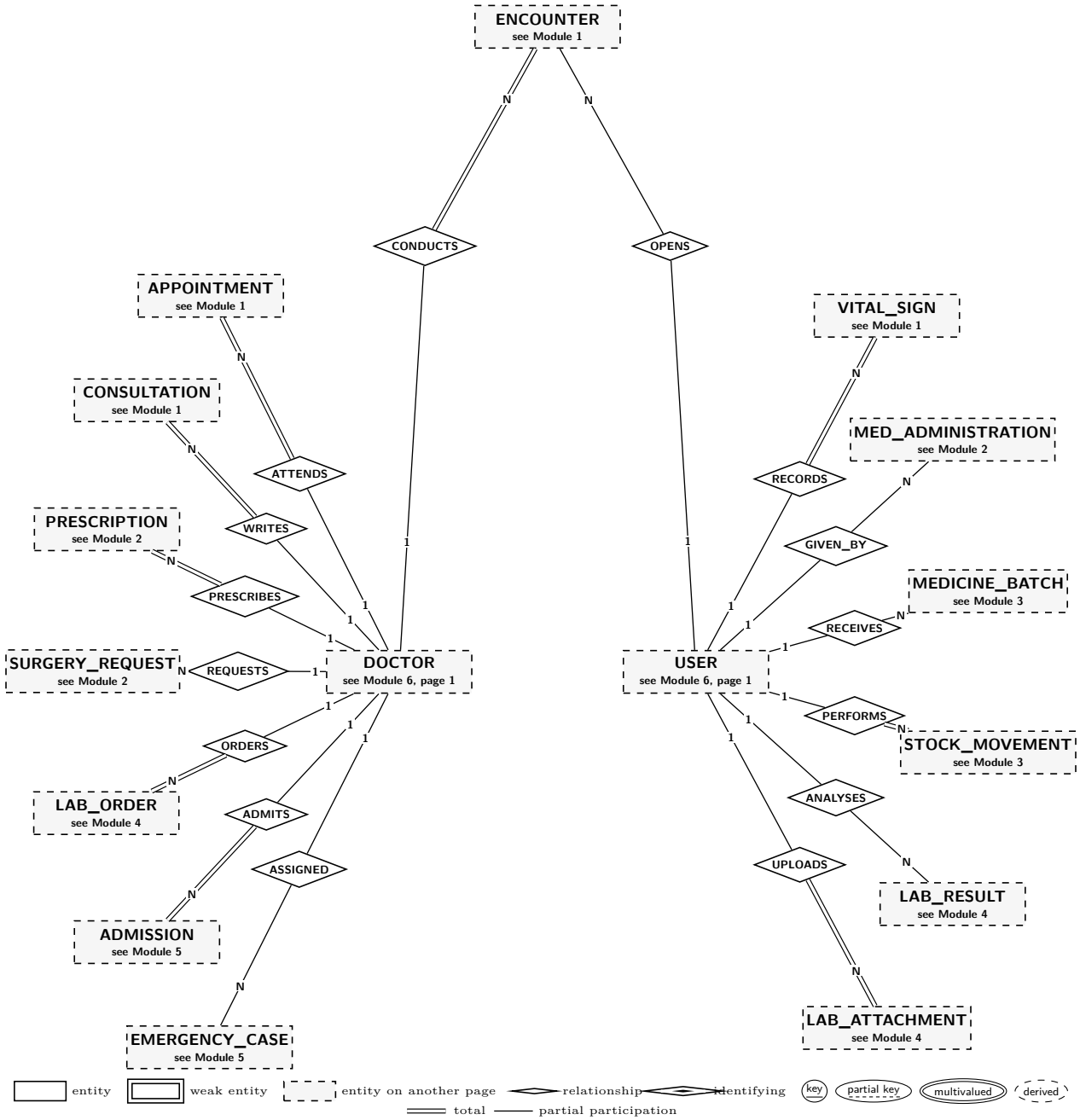


Figure 3.11: Module 6, Staff and audit (page 2 of 2): the records each doctor and each staff account is linked to

DELETE CASCADE).

- **DAILY_ANALYTICS** (owner DOCTOR, partial key analytics_date): one statistics row per doctor per day, UNIQUE (`doctor_id, analytics_date`), deleted with the doctor.

In the tables each also has a surrogate integer key, added during mapping (Section 4.2). Line items such as BILL_ITEM are *not* modelled as weak: nothing in the schema identifies them within their bill, so their own key identifies them.

Module 1, Patients and visits (Figures 3.3, 3.4). PATIENT books APPOINTMENTs and has ENCOUNTERs (visits). An appointment BECOMES at most one encounter and an encounter comes from at most one appointment (1:1, both partial: walk-ins and emergencies have no appointment). VITAL_SIGN readings and CONSULTATION notes always name their patient (total) and are filed under a visit (TAKEN_IN, NOTE_IN) or, for inpatients, an admission (TAKEN_ON, STAY_NOTE). A consultation may also refer to its appointment (NOTE_FOR). Attributes show the derived *age* (computed from *date_of_birth*, never stored), the multivalued *allergies*, and the composite *slot*, *timeline* and *blood_pressure*. Links to other modules: DOCTOR ATTENDS appointments, CONDUCTS encounters and WRITES notes; USER OPENS an encounter and RECORDS vitals (Module 6); PATIENT is ADMITTED, ARRIVES as an emergency, is BILLED, and appears in OCCUPIES (Module 5); PATIENT, ENCOUNTER and CONSULTATION are linked to prescriptions and surgery requests (Module 2) and lab orders (Module 4).

Module 2, Clinical orders (Figure 3.6). A PRESCRIPTION LISTS its PRESCRIPTION_ITEMs (total on the item side). Each item is OF_DRUG one MEDICINE and is FILLED_FROM one or more MEDICINE_BATCHes (M:N, with the relationship attribute *quantity*). Every MED_ADMINISTRATION (a dose on the MAR) is GIVEN_AS part of one item (total) and optionally belongs to an admission and to the nurse who recorded it. A prescription and a SURGERY_REQUEST each belong to a patient and a doctor (total) and optionally to the visit, note or admission they were written in.

Module 3, Pharmacy (Figure 3.5). MEDICINE HAS_BATCH MEDICINE_BATCHes through an identifying relationship. Every STOCK_MOVEMENT MOVES one medicine (total) and, when it touched a batch, comes FROM_BATCH it; movements from before batches existed have none (partial). The medicine's *stock_quantity* is drawn as derived: it is the sum of its batches, stored for speed and kept equal in the same transaction (Section 6.6). USER links: RECEIVES (batches) and PERFORMS (movements).

Module 4, Laboratory (Figure 3.7). A LAB_ORDER is OF_TEST one LAB_TEST, and YIELDS at most one LAB_RESULT (1:1; every result has its order, total). A result has any number of LAB_ATTACHMENTs. Orders link to the patient and doctor (total) and optionally to the visit, note and admission; tests to the DEPARTMENT that OFFERS them; results and files to the USER who ANALYSES and UPLOADS them.

Module 5, Beds and admissions (Figures 3.8, 3.9). A WARD HAS_BED BEDs (identifying). An ADMISSION is ALLOCATED one bed, is ADMITTED for one patient by one doctor (ADMITS) to one department (ADMITTED_TO), all total. OCCUPIES is ternary: each row of the bed history joins one bed, one admission and its patient, with the attributes *assigned_at*, *released_at* and *status*; every admission takes part (total), because a trigger writes the row. An EMERGENCY_CASE may be CARRIED by an AMBULANCE, ASSIGNED a doctor and GIVEN_BED, and RESULTS_IN an admission; all partial, because cases wait in a queue first. Figure 3.9 shows the six records an admission can cover (Modules 1, 2 and 4) and billing: a BILL is BILLED to one patient (total), optionally for an admission, and ITEMISES BILL_ITEMs whose *total_price* is derived (quantity × unit price); the bill's *total_amount* is the stored sum of its items.

Module 6, Staff and audit (Figures 3.10, 3.11). A USER account IS_DOCTOR for at most one DOCTOR (1:1; every doctor has an account). A doctor WORKS_IN one DEPARTMENT and has DOCTOR_SCHEDULE slots and DAILY_ANALYTICS rows (both weak). A WARD_ASSIGNMENT joins a nurse (NURSE_OF, total) to a ward (COVERS, total) for a period, and records the administrator who ASSIGNS it. Every AUDIT_LOG row LOGS the user who acted, when known (failed logins have none). Figure 3.11 gathers, for DOCTOR and USER, every relationship that enters them from Modules 1 to 5, so the two pages of Module 6 together show all 15 links of these two entities.

Chapter 4

ER-to-Relational Mapping

4.1 Mapping Rules

The standard rules were applied, with one project-wide convention: every entity gets a surrogate integer primary key (`AUTO_INCREMENT`), and natural identifiers become UNIQUE keys. Names, phone numbers and batch numbers are not reliable identifiers, and narrow integer keys keep foreign keys and InnoDB's clustered index

small.

1. **Strong entity** → a table with its simple attributes; its key becomes the primary key.
2. **Weak entity** → a table holding the owner's key as a foreign key (NOT NULL) plus the partial key; (owner key, partial key) is a UNIQUE key. A surrogate primary key is added.
3. **1:1** → the foreign key goes on the side that may be missing, with a UNIQUE constraint.
4. **1:N** → the foreign key goes on the N side; NOT NULL when the N side's participation is total.
5. **M:N and ternary** → a new table with a foreign key to each participant and the relationship's attributes.
6. **Composite attribute** → its components become columns. **Multivalued attribute** → a separate table (owner key, value), or (as here, Section 4.2) one column when the value is never searched element by element. **Derived attribute** → not stored, or a generated column, or a stored copy kept up to date.

4.2 Step-by-Step Conversion

ER construct	Relation in hospital_db (real example)
Strong entity	PATIENT → `patients(patient_id, name, date_of_birth, gender, ..., abha_number)`; ABHA becomes UNIQUE `uq_patients_abha`. Likewise for the 25 other strong entities.
Weak entity	BED (owner WARD) → `beds(bed_id, ward_id`^{FK}`, bed_number, ...)` with UNIQUE (`ward_id, bed_number`); MEDICINE_BATCH → `medicine_batches` with UNIQUE (`medicine_id, batch_number`); DOCTOR_SCHEDULE and DAILY_ANALYTICS likewise, with ON DELETE CASCADE from `doctors`.
1:1	BECOMES: `encounters.appointment_id` (NULL for walk-ins) with UNIQUE `uq_encounters_appointment`; IS_DOCTOR: `doctors.user_id` NOT NULL UNIQUE (`uq_doctors_user_id`); YIELDS: `lab_results.order_id` NOT NULL UNIQUE.
1:N	BOOKS: `appointments.patient_id` NOT NULL (total); CARRIES: `emergency_cases.ambulance_id` NULL (partial). 64 relationships are mapped this way.
M:N with attribute	FILLED_FROM → `prescription_item_batches(id, item_id`^{FK}`, batch_id`^{FK}`, quantity)`, UNIQUE (`item_id, batch_id`).
Ternary with attributes	OCCUPIES → `bed_assignment_log(log_id, bed_id`^{FK}`, patient_id`^{FK}`, admission_id`^{FK}`, assigned_at, released_at, status)`.
Composite	*blood_pressure* → `bp_systolic, bp_diastolic` (CHECK: both or neither, systolic > diastolic); *slot* → `appointment_date, appointment_time`.
Multivalued	*allergies* is stored as one text column `patients.allergies`: the application only displays it on the chart and never queries one allergy, so a separate `patient_allergies` table was not created (a limitation, Section 11.2).
Derived	*age*: not stored, computed as `TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE())`; *total_price*: generated column `quantity * unit_price`; *stock_quantity*, *total_amount*, *current_workload*: stored copies kept equal by transactions and triggers (Section 6.6).

4.3 Final Relational Schema

All 33 relations, with columns in table order. Underlined: primary key. *Italic*^{FK}: foreign key (with its target when the name does not say it). ^{G}: generated column. UK: other candidate keys (UNIQUE). Not shown: the 15 composite consistency keys of Section 6.5, which re-check pairs of these foreign keys.

patients(patient_id, name, date_of_birth, dob_estimated, gender, blood_group, phone, address, emergency_contact, created_at, updated_at, allergies, abha_number) UK (abha_number)

appointments(appointment_id, *patient_id*^{FK}, *doctor_id*^{FK}, appointment_date, appointment_time, status, reason, notes, created_at, updated_at) UK (doctor_id, appointment_date, appointment_time)

encounters(encounter_id, *patient_id*^{FK}, *doctor_id*^{FK}, *appointment_id*^{FK}, *admission_id*^{FK}, encounter_type, status, arrived_at, triaged_at, start_timestamp, end_timestamp, *created_by*$^{FK\to users}$, backfilled_from, created_at, updated_at) UK (appointment_id); (backfilled_from)

consultations(consultation_id, *appointment_id*^{FK}, *patient_id*^{FK}, *doctor_id*^{FK}, *admission_id*^{FK}, symptoms, diagnosis, assessment, plan, notes, consultation_time, created_at, *encounter_id*^{FK})

vital_signs(vital_id, *patient_id*^{FK}, *encounter_id*^{FK}, *admission_id*^{FK}, recorded_at_utc, *recorded_by*$^{FK\to users}$, temperature_c, pulse_bpm, resp_rate, bp_systolic, bp_diastolic, spo2_pct, pain_score, notes, created_at)

surgery_requests(request_id, *patient_id*^{FK}, *doctor_id*^{FK}, procedure_name, diagnosis, priority, requested_date, status, notes, created_at, updated_at, *encounter_id*^{FK})

departments(department_id, name, description, floor, phone, is_active) UK (name)

wards(ward_id, name, *department_id*^{FK}, floor, capacity, ward_type)

beds(bed_id, *ward_id*^{FK}, bed_number, bed_type, status, has_ventilator) UK (ward_id, bed_number)

admissions(admission_id, *patient_id*^{FK}, *doctor_id*^{FK}, *bed_id*^{FK}, *department_id*^{FK}, *emergency_id*^{FK}, admission_date, discharge_date, status, diagnosis, notes, created_at, updated_at)

bed_assignment_log(log_id, *bed_id*^{FK}, *patient_id*^{FK}, *admission_id*^{FK}, assigned_at, released_at, status)

emergency_cases(emergency_id, *patient_id*^{FK}, *ambulance_id*^{FK}, severity, symptoms, arrival_time, required_specialization, required_bed_type, ventilator_required, status, *assigned_doctor_id*$^{\text{FK}\to\text{doctors}}$, *assigned_bed_id*$^{\text{FK}\to\text{beds}}$, notes, created_at, updated_at)

ambulances(ambulance_id, vehicle_number, driver_name, driver_phone, current_location, status) UK (vehicle_number)

medicines(medicine_id, name, category, manufacturer, stock_quantity, reorder_level, unit_price, expiry_date, requires_prescription, created_at, updated_at)

medicine_batches(batch_id, *medicine_id*^{FK}, batch_number, quantity, expiry_date, received_date, *received_by*$^{\text{FK}\to\text{users}}$, created_at, updated_at) UK (medicine_id, batch_number)

prescriptions(prescription_id, *consultation_id*^{FK}, *patient_id*^{FK}, *doctor_id*^{FK}, prescription_date, status, notes, created_at, *encounter_id*^{FK}, *admission_id*^{FK})

prescription_items(item_id, *prescription_id*^{FK}, *medicine_id*^{FK}, dosage, frequency, duration, quantity, dispensed, dispensed_at, frequency_code, duration_days, route, units_per_dose)

prescription_item_batches(id, *item_id*^{FK}, *batch_id*^{FK}, quantity, created_at) UK (item_id, batch_id)

pharmacy_stock_movements(movement_id, *medicine_id*^{FK}, movement_type, quantity, reason, notes, *performed_by*$^{\text{FK}\to\text{users}}$, created_at, reference_id, *batch_id*^{FK})

medication_administrations(administration_id, *prescription_item_id*$^{\text{FK}\to\text{prescription_items}}$, *admission_id*^{FK}, scheduled_at_utc, status, administered_at_utc, dose_given, route_given, reason, *recorded_by*$^{\text{FK}\to\text{users}}$, recorded_at_utc, created_at, updated_at) UK (prescription_item_id, scheduled_at_utc)

lab_tests(test_id, name, category, *department_id*^{FK}, normal_range, unit, cost, reference_low, reference_high, critical_low, critical_high)

lab_orders(order_id, *consultation_id*^{FK}, *patient_id*^{FK}, *doctor_id*^{FK}, *test_id*^{FK}, order_date, status, priority, notes, created_at, updated_at, *encounter_id*^{FK}, *admission_id*^{FK})

lab_results(result_id, *order_id*^{FK}, result_value, unit, reference_range, interpretation, result_date, technician_notes, *performed_by*$^{\text{FK}\to\text{users}}$, reviewed_by_doctor, reviewed_at, created_at, numeric_value, reference_low, reference_high, interpretation_source) UK (order_id)

lab_result_attachments(attachment_id, *result_id*^{FK}, original_filename, stored_filename, mime_type, file_size, *uploaded_by*$^{\text{FK}\to\text{users}}$, uploaded_at) UK (stored_filename)

users(user_id, username, password_hash, role, full_name, email, phone, is_active, created_at, updated_at) UK (email); (username)

doctors(doctor_id, *user_id*^{FK}, *department_id*^{FK}, name, specialization, phone, status, shift, current_workload, max_workload, created_at, updated_at) UK (user_id)

doctor_schedules(schedule_id, *doctor_id*^{FK}, day_of_week, start_time, end_time, is_available) UK (doctor_id, day_of_week, start_time)

doctor_daily_analytics(analytics_id, *doctor_id*^{FK}, analytics_date, patients_seen, emergency_cases, appointment_patients, patients_treated, cases_solved, cases_postponed, cases_cancelled, cases_pending, created_at, updated_at) UK (doctor_id, analytics_date)

nurse_ward_assignments(assignment_id, *nurse_user_id*$^{\text{FK}\to\text{users}}$, *ward_id*^{FK}, start_date, end_date, *assigned_by*$^{\text{FK}\to\text{users}}$, created_at, updated_at)

bills(bill_id, *patient_id*^{FK}, *admission_id*^{FK}, bill_date, total_amount, paid_amount, status, payment_method, notes, created_at, updated_at)

bill_items(item_id, *bill_id*^{FK}, description, category, quantity, unit_price, total_price^{G}, created_at)

audit_logs(audit_id, *user_id*^{FK}, role, action, entity_type, entity_id, patient_id, outcome, status_code, method, path, ip_address, details, created_at)

schema_migrations(version, applied_at, note)

Chapter 5

Functional Dependencies

5.1 Identification of Functional Dependencies

A functional dependency $X \to Y$ holds when any two rows with the same X have the same Y. Each one listed here comes from a business rule that the schema or the code enforces: a primary or UNIQUE key, a foreign key whose value is copied from its parent when a row is written, or a computed value. A pattern that merely happens to hold in today's rows is not listed. For a nullable left side (for example `encounter_id`), the

dependency applies to the rows where it is not NULL, as in SQL's treatment of UNIQUE. Examples:

$$\textit{patient_id} \rightarrow \textit{name, date_of_birth, gender, blood_group, phone, \ldots, abha_number}$$

$$\textit{doctor_id, appointment_date, appointment_time} \rightarrow \textit{appointment_id}$$

$$\textit{encounter_id} \rightarrow \textit{patient_id, doctor_id} \quad (\text{in } \texttt{lab_orders})$$

5.2 Candidate Keys and Attribute Classification

Table 5.1 gives, for every relation, its dependencies, its candidate keys, its prime attributes (those in some candidate key) and the number of non-prime attributes (all the others). A UNIQUE column that allows NULL (such as `abha_number`) identifies a row only when present; it is listed as a key because MySQL enforces it.

Table 5.1: Functional dependencies, candidate keys and attribute classification

Relation	Functional dependencies	Candidate keys	Prime attributes	Non-prime
patients	patient_id → all attributes; abha_number → patient_id (when not NULL)	{patient_id}, {abha_number}	patient_id, abha_number	11
appointments	appointment_id → all attributes; (doctor_id, appointment_date, appointment_time) → appointment_id	{appointment_id}, {doctor_id, appointment_date, appointment_time}	appointment_id, doctor_id, appointment_date, appointment_time	6
encounters	encounter_id → all attributes; appointment_id → all attributes (when not NULL); admission_id → patient_id (when not NULL)	{encounter_id}, {appointment_id}, {backfilled_from}	encounter_id, appointment_id, backfilled_from	12
consultations	consultation_id → all attributes; encounter_id → patient_id, doctor_id; appointment_id → patient_id, doctor_id, encounter_id; admission_id → patient_id	{consultation_id}	consultation_id	12
vital_signs	vital_id → all attributes; encounter_id → patient_id; admission_id → patient_id	{vital_id}	vital_id	14
surgery_requests	request_id → all attributes; encounter_id → patient_id, doctor_id	{request_id}	request_id	11
departments	department_id → all attributes; name → department_id	{department_id}, {name}	department_id, name	4
wards	ward_id → all attributes	{ward_id}	ward_id	5
beds	bed_id → all attributes; (ward_id, bed_number) → bed_id	{bed_id}, {ward_id, bed_number}	bed_id, ward_id, bed_number	3
admissions	admission_id → all attributes	{admission_id}	admission_id	12
bed_assignment_log	log_id → all attributes; admission_id → all attributes	{log_id}	log_id	6
emergency_cases	emergency_id → all attributes	{emergency_id}	emergency_id	14
ambulances	ambulance_id → all attributes; vehicle_number → ambulance_id	{ambulance_id}, {vehicle_number}	ambulance_id, vehicle_number	4
medicines	medicine_id → all attributes	{medicine_id}	medicine_id	10
medicine_batches	batch_id → all attributes; (medicine_id, batch_number) → batch_id	{batch_id}, {medicine_id, batch_number}	batch_id, medicine_id, batch_number	6
prescriptions	prescription_id → all attributes; encounter_id → patient_id, doctor_id; admission_id → patient_id; consultation_id → patient_id	{prescription_id}	prescription_id	9
prescription_items	item_id → all attributes; dispensed_at → dispensed	{item_id}	item_id	12
prescription_item_batches	id → all attributes; (item_id, batch_id) → id	{id}, {item_id, batch_id}	id, item_id, batch_id	2
pharmacy_stock_movements	movement_id → all attributes; batch_id → medicine_id	{movement_id}	movement_id	9
medication_administrations	administration_id → all attributes; (prescription_item_id, scheduled_at_utc) → administration_id	{administration_id}, {prescription_item_id, scheduled_at_utc}	administration_id, prescription_item_id, scheduled_at_utc	10
lab_tests	test_id → all attributes	{test_id}	test_id	10
lab_orders	order_id → all attributes; encounter_id → patient_id, doctor_id; admission_id → patient_id; consultation_id → patient_id	{order_id}	order_id	12
lab_results	result_id → all attributes; order_id → result_id	{result_id}, {order_id}	result_id, order_id	14
lab_result_attachments	attachment_id → all attributes; stored_filename → attachment_id	{attachment_id}, {stored_filename}	attachment_id, stored_filename	6
users	user_id → all attributes; username → user_id; email → user_id	{user_id}, {email}, {username}	user_id, email, username	7
doctors	doctor_id → all attributes; user_id → doctor_id	{doctor_id}, {user_id}	doctor_id, user_id	10
doctor_schedules	schedule_id → all attributes; (doctor_id, day_of_week, start_time) → schedule_id	{schedule_id}, {doctor_id, day_of_week, start_time}	schedule_id, doctor_id, day_of_week, start_time	2
doctor_daily_analytics	analytics_id → all attributes; (doctor_id, analytics_date) → analytics_id	{analytics_id}, {doctor_id, analytics_date}	analytics_id, doctor_id, analytics_date	10
nurse_ward_assignments	assignment_id → all attributes	{assignment_id}	assignment_id	7
bills	bill_id → all attributes; admission_id → patient_id	{bill_id}	bill_id	10
bill_items	item_id → all attributes	{item_id}	item_id	7

Relation	Functional dependencies	Candidate keys	Prime attributes	Non-prime
audit_logs	audit_id → all attributes	{audit_id}	audit_id	13
schema_migrations	version → applied_at, note	{version}	version	2

5.3 FD Analysis

- **Keys.** Each primary key determines its row. Each UNIQUE key is a business identifier with its own dependency: a doctor's slot (`doctor_id, appointment_date, appointment_time`) identifies one appointment, because a doctor cannot be in two places; a ward and bed number identify one bed; a medicine and supplier batch number identify one batch; an order identifies its single result.
- **Copied context.** `lab_orders`, `prescriptions`, `consultations`, `surgery_requests` and `vital_signs` store the patient (and doctor) of the visit, admission or note they are filed under. The code copies these from the parent and checks them (`resolveEncounterForRecord`, `resolveOrderContext`), and composite foreign keys now enforce them, so `encounter_id` → `patient_id, doctor_id` is a real dependency in those tables. In `pharmacy_stock_movements`, `batch_id` → `medicine_id` for the same reason.
- **Flags.** In `prescription_items` the single dispensing statement sets `dispensed_at = NOW()` and `dispensed = TRUE` together, so `dispensed_at` → `dispensed`.
- **Removed by normalization.** Three dependencies held in the schema before this work and were removed by decomposition (Chapter 6): `ward_id` → `floor` in `beds`, `doctor_id` → `department_id` in `appointments`, `prescription_item_id` → `patient_id` in `medication_administrations`; a fourth, (`quantity, unit_price`) → `total_price` in `bill_items`, now holds by definition of a generated column.
- **Not dependencies.** Some patterns look like dependencies but are not rules, so they are not listed: `wards.department_id` ↛ `floor` (a department may span floors; each ward's floor is entered); `admissions.bed_id` ↛ `department_id` (staff choose the department on the form); `emergency_cases.required_bed_type` ↛ `ventilator_required` (an ICU case may or may not need one); `audit_logs.user_id` ↛ `role` (a seed migration logs the role MIGRATION); and the order's admission is a snapshot taken when the order was placed, not a function of its visit.

Chapter 6

Database Normalization

Normalization is shown in two parts. Sections 6.1 to 6.5 take one outpatient record, as a clerk might keep it on paper, through each normal form to the tables this system has. The patient, doctor, department, medicines and tests in it are real rows of the seed data; the visit itself is illustrative. The same sections then show the four decompositions performed on the live schema (migrations 015–022) at the normal form each one fixed. Section 6.6 gives the final schema and the normal form of all 33 tables.

6.1 Unnormalized Form (UNF)

Table 6.1: One outpatient visit as a single unnormalized record

Patient	Phone	Doctor	Dept	Visit	Medicines (dose, freq, qty)	Tests (cost, result)
Aarav Sharma, M, 1985-03-15	9111000001	Dr. John Smith	Cardiology, floor 2	2026-10-13 09:30; chest tightness / suspected angina	Aspirin 75mg (75 mg, OD, 30); Atorvastatin 10mg (10 mg, OD, 30)	Lipid Profile (800.00, pending); Blood Glucose Fasting (150.00, 128 mg/dL, HIGH)

Problems. *Medicines* and *Tests* are repeating groups (a list in one cell), *Patient*, *Dept* and *Visit* each pack several facts into one cell, and there is no key.

6.2 First Normal Form (1NF)

Each cell is split into atomic values and each repeated item into its own row. The record becomes one row per (medicine, test) pair of the visit, four rows in all:

VISIT_RECORD(<u>patient_name, visit_time, medicine, test</u>, dob, gender, phone, doctor, department, dept_floor, symptoms, diagnosis, dose, frequency, quantity, test_cost, result_value, interpretation)

It is in 1NF but every patient and visit fact is repeated four times, giving the three anomalies: **update** (a new phone number must be written in four rows), **insertion** (a test cannot be recorded for a visit without a

medicine) and **deletion** (removing the last medicine loses the diagnosis). Every live table is in 1NF: all columns are single-valued and typed, and lists such as a prescription's lines are rows of their own table.

6.3 Second Normal Form (2NF)

The record. Partial dependencies on the key (*patient, visit time, medicine, test*): *patient* → dob, gender, phone; (*patient, visit time*) → doctor, department, dept_floor, symptoms, diagnosis; (*visit, medicine*) → dose, frequency, quantity; (*visit, test*) → result; *test* → cost. Removing them gives:

PATIENT(patient, dob, gender, phone) VISIT(patient, visit_time, doctor, department, dept_floor, symptoms, diagnosis)
VISIT_MEDICINE(patient, visit_time, medicine, dose, frequency, quantity) VISIT_TEST(patient, visit_time, test, result)
TEST(test, cost)

A surrogate key is then given to each entity (`patient_id`, `encounter_id`, ...) because names are not unique.

The live schema: three 2NF violations decomposed. Each was a non-key column depending on part of a composite candidate key. In each case the determinant's own table already held the value, so the decomposition removes the copy and reads it through the existing foreign key; no new table was needed.

1. **A1, `beds`.** Key (`ward_id, bed_number`); `ward_id` → `floor`.
   `beds(bed_id, ward_id, bed_number, ~~floor~~, bed_type, status, has_ventilator)` ⇒ `beds(bed_id, ward_id, bed_number, bed_type, status, has_ventilator)` ⋈ `wards(ward_id, ..., floor)`.
2. **A2, `appointments`.** Key (`doctor_id, appointment_date, appointment_time`); `doctor_id` → `department_id` (copied from the booked doctor). ⇒ `appointments` without `department_id` ⋈ `doctors(doctor_id, department_id)`; the view `v_appointments` returns the old columns. Four seed rows booked under another department were first set to their doctor's department (old values kept in the audit log).
3. **A3, `medication_administrations`.** Key (`prescription_item_id, scheduled_at_utc`); `prescription_item_id` → `patient_id`. ⇒ the dose table without `patient_id` ⋈ `prescription_items` ⋈ `prescriptions(prescription_id, patient_id)`.

Each decomposition is **lossless**, because the shared attribute is the key of the other relation and every row was checked to rebuild from the join before the column was dropped (35/35 beds, 9/9 appointments, 15/15 doses). It is also **dependency-preserving**, because the removed dependency is now enforced by the other relation's key.

6.4 Third Normal Form (3NF)

The record. In VISIT, department and dept_floor depend on the doctor, not the visit (*visit* → *doctor* → *department* → *floor*). Removing the transitive dependencies:

VISIT(visit, patient, doctor, visit_time, symptoms, diagnosis) DOCTOR(doctor, name, department) DEPARTMENT(department, name, floor)

A visit can have several notes written by its doctor, so symptoms and diagnosis move to a note of the visit (`consultations`); medicines become a catalogue (`medicines`) and what is prescribed is grouped under a prescription header.

The live schema: one 3NF violation decomposed. A5, `bill_items`: (`quantity, unit_price`) → `total_price`, a dependency through two non-key columns. The column became `total_price DECIMAL(10,2) GENERATED ALWAYS AS (quantity * unit_price) VIRTUAL`: it is computed when read, so it cannot disagree, and the API still returns it (19/19 existing rows already matched).

6.5 Boyce–Codd Normal Form (BCNF)

The record. After 3NF every determinant of the record's relations is a key: a prescription line (*prescription, medicine*) gets its own id, a lab result is determined by its order, and a test's cost by the test. The result is the tables of this system: PATIENT → `patients`; DOCTOR, DEPARTMENT → `doctors`, `departments`; VISIT → `appointments` (the booking) and `encounters` (the visit); note → `consultations`; VISIT_MEDICINE → `prescriptions` + `prescription_items`; TEST, VISIT_TEST → `lab_tests`, `lab_orders` + `lab_results`.

The live schema: tables checked against BCNF. 24 of the 33 tables are in BCNF: every non-trivial dependency has a candidate key on its left (Table 5.1). Nine are not, and are kept on purpose:

- **Eight copy a patient, doctor or medicine from an optional parent** (Table 6.3), e.g. `lab_orders`: `encounter_id` → `patient_id, doctor_id`. The textbook decomposition (drop `patient_id`, read it through `encounter_id`) is *lossy* here: the link is optional, and an order with no visit would lose its patient (3 of the 9 live orders have neither a visit nor an admission). The doctor's access check also filters on these columns in every table. So the copies stay, and **the database enforces them**: each parent got a UNIQUE key over its id and the copied columns (e.g. `encounters (encounter_id, patient_id, doctor_id)`) and each child a composite foreign key to it (`fk_ctx_*`, 15 in all, migration 018). InnoDB checks a composite foreign key

only when none of its columns is NULL, so rows without the link are unaffected; a copy that disagrees is refused (test T10, Chapter 10).

- `prescription_items`: `dispensed_at` → `dispensed`. Replacing the flag by "`dispensed_at IS NOT NULL`" would lose information: three seed items are dispensed with no recorded time, and the guard against dispensing twice keys on the flag. The dependency is enforced by `CHECK (dispensed_at IS NULL OR dispensed = 1)` (migration 023).

6.6 Final Normalized Schema

The final relations are those of Section 4.3. Table 6.2 gives each table's highest normal form before this work (migration 014) and now; Table 6.3 lists the nine tables outside BCNF and what keeps their copies consistent. In summary: 20 tables were in BCNF before and 24 are now, every table is in 2NF (the three 2NF violations were decomposed), and the nine remaining are in 2NF with each dependency enforced by the database.

Table 6.2: Normal form of every table, before (migration 014) and now

Table	Before	Now	Note	Table	Before	Now	Note
patients	BCNF	BCNF	keys only	prescription_item_batches	BCNF	BCNF	keys only
appointments	1NF	BCNF	decomposed (A2)	pharmacy_stock_movements	2NF	2NF	kept; Table 6.3
encounters	2NF	2NF	kept; Table 6.3	medication_administrations	1NF	BCNF	decomposed (A3)
consultations	2NF	2NF	kept; Table 6.3	lab_tests	BCNF	BCNF	keys only
vital_signs	2NF	2NF	kept; Table 6.3	lab_orders	2NF	2NF	kept; Table 6.3
surgery_requests	2NF	2NF	kept; Table 6.3	lab_results	BCNF	BCNF	keys only
departments	BCNF	BCNF	keys only	lab_result_attachments	BCNF	BCNF	keys only
wards	BCNF	BCNF	keys only	users	BCNF	BCNF	keys only
beds	1NF	BCNF	decomposed (A1)	doctors	BCNF	BCNF	keys only
admissions	BCNF	BCNF	keys only	doctor_schedules	BCNF	BCNF	keys only
bed_assignment_log	BCNF	BCNF	keys only	doctor_daily_analytics	BCNF	BCNF	keys only
emergency_cases	BCNF	BCNF	keys only	nurse_ward_assignments	BCNF	BCNF	keys only
ambulances	BCNF	BCNF	keys only	bills	2NF	2NF	kept; Table 6.3
medicines	BCNF	BCNF	keys only	bill_items	2NF	BCNF	decomposed (A5)
medicine_batches	BCNF	BCNF	keys only	audit_logs	BCNF	BCNF	keys only
prescriptions	2NF	2NF	kept; Table 6.3	schema_migrations	BCNF	BCNF	keys only
prescription_items	2NF	2NF	kept; Table 6.3				

Table 6.3: The nine tables kept outside BCNF: dependency and how consistency is enforced

Table	Dependency (left side not a key)	Enforced by
encounters	admission_id → patient_id (when not NULL)	fk_ctx_encounters_admission (admission_id, patient_id) → admissions
consultations	encounter_id → patient_id, doctor_id appointment_id → patient_id, doctor_id, encounter_id admission_id → patient_id	fk_ctx_consultations_encounter (encounter_id, patient_id, doctor_id) → encounters fk_ctx_consultations_appointment (appointment_id, patient_id, doctor_id) → appointments fk_ctx_consultations_admission (admission_id, patient_id) → admissions API: POST /consultations answers 400 when the appointment is another patient's or doctor's
vital_signs	encounter_id → patient_id admission_id → patient_id	fk_ctx_vital_signs_encounter (encounter_id, patient_id) → encounters fk_ctx_vital_signs_admission (admission_id, patient_id) → admissions API: POST /vitals answers 400 when the visit or admission is another patient's
surgery_requests	encounter_id → patient_id, doctor_id	fk_ctx_surgery_requests_encounter (encounter_id, patient_id, doctor_id) → encounters
prescriptions	encounter_id → patient_id, doctor_id admission_id → patient_id consultation_id → patient_id	fk_ctx_prescriptions_encounter (encounter_id, patient_id, doctor_id) → encounters fk_ctx_prescriptions_admission (admission_id, patient_id) → admissions fk_ctx_prescriptions_consultation (consultation_id, patient_id) → consultations API: POST /prescriptions answers 400 when the consultation is another patient's or doctor's
prescription_items	dispensed_at → dispensed	chk_prescription_items_dispensed: CHECK (dispensed_at IS NULL OR dispensed = 1) (migration 023) dispenseService, the only writer of dispensed_at, sets both columns in one UPDATE (SET dispensed = TRUE, dispensed_at = NOW() AND dispensed = FALSE)
pharmacy_stock_movements	batch_id → medicine_id	fk_ctx_stock_movements_batch (batch_id, medicine_id) → medicine_batches
lab_orders	encounter_id → patient_id, doctor_id admission_id → patient_id consultation_id → patient_id	fk_ctx_lab_orders_encounter (encounter_id, patient_id, doctor_id) → encounters fk_ctx_lab_orders_admission (admission_id, patient_id) → admissions fk_ctx_lab_orders_consultation (consultation_id, patient_id) → consultations

Table	Dependency	Enforced by
bills	admission_id → patient_id	fk_ctx_bills_admission (admission_id, patient_id) → admissions API: POST /bills answers 400 when the admission is another patient's, or does not exist

Values derived from *other* tables do not break BCNF inside a table, but they are redundancy, and each is kept equal by the database: `medicines.stock_quantity` (sum of its batches, updated in the dispensing and stock transactions), `bills.total_amount` (sum of its items), `doctors.current_workload` and `beds.status` (maintained by the admission triggers), and `appointments.status` (mirrors its visit, written in the same transaction).

Chapter 7

Relational Integrity and Constraints

The schema declares 33 primary keys, 87 foreign keys (72 relationships and 15 composite consistency keys), 23 UNIQUE keys and 30 CHECK constraints, and runs in strict SQL mode, so an invalid value is an error rather than a silent conversion. Seven triggers keep derived values and the audit log correct. Table 7.1 gives a real example of each kind.

Table 7.1: Integrity constraints in hospital_db, with real examples

Constraint	Example	What it prevents
Domain	`patients.gender ENUM('MALE','FEMALE','OTHER')`; `bills.total_amount DECIMAL(12,2)`; `appointment_date DATE`	Values outside the column's type or list ("M", "tomorrow")
Primary key	`patients(patient_id)`; every table has one	Duplicate or missing row identity
Foreign key	`lab_orders.test_id → lab_tests(test_id)`	An order for a test that does not exist (test T3)
Composite FK	`lab_orders(encounter_id, patient_id, doctor_id) → encounters(encounter_id, patient_id, doctor_id)`	An order whose patient differs from its visit's patient (Section 6.5)
UNIQUE	`users.username`; `uq_patients_abha`; `(doctor_id, appointment_date, appointment_time)`	Two accounts with one username (T2); one ABHA number for two patients (T6); a double booking (T13)
NOT NULL	`appointments.patient_id`; `admissions.bed_id`; `lab_results.order_id`	An appointment for nobody; an admission without a bed (total participation, Chapter 3)
CHECK	`medicines_chk_1 (stock_quantity >= 0)`; `bills_chk_2 (paid_amount <= total_amount)`; `chk_vitals_spo2 (spo2_pct BETWEEN 50 AND 100)`; `chk_patients_abha_format` (14 digits)	Negative stock (T4), over-payment, impossible readings, malformed ABHA
Generated column	`bill_items.total_price AS (quantity * unit_price)`	A line total that disagrees with its quantity and price
Trigger	`before_medicine_stock_update` (SIGNAL if negative); `audit_logs_no_update`, `audit_logs_no_delete`	Negative stock by any path; editing or deleting audit rows (T12)

Referential actions. Every ON UPDATE rule is NO ACTION (InnoDB treats it as RESTRICT): primary keys are surrogates and never change. The ON DELETE rule of each foreign key follows the meaning of the relationship:

- **RESTRICT (67 keys)** for records that must not lose their parent: a patient with admissions, visits or bills cannot be deleted (`DELETE FROM patients WHERE patient_id = 1` fails with error 1451, Section 8.4). The application has no delete operation for patients, and staff accounts are deactivated (`users.is_active`), never deleted.
- **CASCADE (12)** for parts that have no meaning without their whole: `bill_items` with their `bills`, `prescription_items` with their `prescriptions`, `lab_results` with their `lab_orders`, `doctor_schedules` and `doctor_daily_analytics` with their doctor (the weak entities).
- **SET NULL (8)** for optional references whose loss leaves the child valid: an emergency case keeps its record when its ambulance (`ambulance_id`), doctor or bed is removed; a lab result keeps its value when the technician's account is removed (`performed_by`).

Together these keep the data consistent without relying on the application: a row cannot reference a missing row, a copy cannot disagree with its source, totals cannot go out of range, and the history in the audit log cannot be rewritten.

Chapter 8

MySQL Database Implementation

All outputs in this chapter are real: they were produced on `hospital_db_test`, a database built from the seed data and every migration (same schema as `hospital_db`, checked by a schema-diff script). Statements that change data were run inside a transaction that was then rolled back.

8.1 Database Creation

The schema is built by numbered migration files (`database/migrations/000` to `023`), applied in order by `server/scripts/migrate.js`, which backs up the database first and records each file in `schema_migrations`. The database and the restricted account the application runs as:

Listing 8.1: Creating the database and the application's restricted account

```
CREATE DATABASE hospital_db CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
CREATE USER 'hospital_app'@'127.0.0.1' IDENTIFIED BY RANDOM PASSWORD;
GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE ON `hospital_db`.* TO 'hospital_app'@'127.0.0.1';
```

The application account cannot create, alter or drop tables or triggers; migrations use a separate administrative account.

8.2 Table Creation

Six key tables are shown below, exactly as MySQL reports them (`SHOW CREATE TABLE`); all 33 are in the complete script (Appendix A).

Listing 8.2: `patients`: unique ABHA number with a format CHECK

```
CREATE TABLE `patients` (
  `patient_id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) NOT NULL,
  `date_of_birth` date NOT NULL,
  `dob_estimated` tinyint(1) NOT NULL DEFAULT '0',
  `gender` enum('MALE','FEMALE','OTHER') NOT NULL,
  `blood_group` enum('A+','A-','B+','B-','AB+','AB-','O+','O-') DEFAULT NULL,
  `phone` varchar(20) DEFAULT NULL,
  `address` text,
  `emergency_contact` varchar(100) DEFAULT NULL,
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `allergies` text,
  `abha_number` char(14) DEFAULT NULL,
  PRIMARY KEY (`patient_id`),
  UNIQUE KEY `uq_patients_abha` (`abha_number`),
  KEY `idx_patients_name` (`name`),
  KEY `idx_patients_phone` (`phone`),
  CONSTRAINT `chk_patients_abha_format` CHECK (((`abha_number` is null) or regexp_like(`abha_number`,_utf8mb4'^[0-9]{14}$')))
) ENGINE=InnoDB;
```

Listing 8.3: `encounters`: the visit, with its 1:1 link to an appointment and the parent key used by consistency keys

```
CREATE TABLE `encounters` (
  `encounter_id` int NOT NULL AUTO_INCREMENT,
  `patient_id` int NOT NULL,
  `doctor_id` int NOT NULL,
  `appointment_id` int DEFAULT NULL,
  `admission_id` int DEFAULT NULL,
  `encounter_type` enum('OUTPATIENT','INPATIENT','EMERGENCY') NOT NULL DEFAULT 'OUTPATIENT',
  `status` enum('PLANNED','ARRIVED','TRIAGED','IN_PROGRESS','FINISHED','CANCELLED','ENTERED_IN_ERROR') NOT NULL,
  `arrived_at` datetime DEFAULT NULL,
  `triaged_at` datetime DEFAULT NULL,
  `start_timestamp` datetime DEFAULT NULL,
  `end_timestamp` datetime DEFAULT NULL,
  `created_by` int DEFAULT NULL,
  `backfilled_from` varchar(40) DEFAULT NULL,
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`encounter_id`),
  UNIQUE KEY `uq_encounters_id_patient_doctor` (`encounter_id`,`patient_id`,`doctor_id`),
  UNIQUE KEY `uq_encounters_id_patient` (`encounter_id`,`patient_id`),
  UNIQUE KEY `uq_encounters_appointment` (`appointment_id`),
  UNIQUE KEY `uq_encounters_backfilled_from` (`backfilled_from`),
```

```
  KEY `idx_encounters_doctor_status` (`doctor_id`,`status`),
  KEY `idx_encounters_patient_status` (`patient_id`,`status`),
  KEY `idx_encounters_admission` (`admission_id`),
  KEY `fk_encounters_created_by` (`created_by`),
  KEY `fk_ctx_encounters_admission` (`admission_id`,`patient_id`),
  CONSTRAINT `fk_ctx_encounters_admission` FOREIGN KEY (`admission_id`, `patient_id`) REFERENCES `admissions` (`admission_id`, `
      patient_id`) ON DELETE RESTRICT,
  CONSTRAINT `fk_encounters_admission` FOREIGN KEY (`admission_id`) REFERENCES `admissions` (`admission_id`) ON DELETE RESTRICT,
  CONSTRAINT `fk_encounters_appointment` FOREIGN KEY (`appointment_id`) REFERENCES `appointments` (`appointment_id`) ON DELETE
      RESTRICT,
  CONSTRAINT `fk_encounters_created_by` FOREIGN KEY (`created_by`) REFERENCES `users` (`user_id`) ON DELETE RESTRICT,
  CONSTRAINT `fk_encounters_doctor` FOREIGN KEY (`doctor_id`) REFERENCES `doctors` (`doctor_id`) ON DELETE RESTRICT,
  CONSTRAINT `fk_encounters_patient` FOREIGN KEY (`patient_id`) REFERENCES `patients` (`patient_id`) ON DELETE RESTRICT,
  CONSTRAINT `chk_encounters_times` CHECK (((`end_timestamp` is null) or (`start_timestamp` is null) or (`end_timestamp` >= `
      start_timestamp`)))
) ENGINE=InnoDB;
```

Listing 8.4: `admissions`: five foreign keys with RESTRICT, SET NULL and a composite parent key

```
CREATE TABLE `admissions` (
  `admission_id` int NOT NULL AUTO_INCREMENT,
  `patient_id` int NOT NULL,
  `doctor_id` int NOT NULL,
  `bed_id` int NOT NULL,
  `department_id` int NOT NULL,
  `emergency_id` int DEFAULT NULL,
  `admission_date` datetime NOT NULL,
  `discharge_date` datetime DEFAULT NULL,
  `status` enum('ACTIVE','DISCHARGED','TRANSFERRED') DEFAULT 'ACTIVE',
  `diagnosis` text NOT NULL,
  `notes` text,
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`admission_id`),
  UNIQUE KEY `uq_admissions_id_patient` (`admission_id`,`patient_id`),
  KEY `department_id` (`department_id`),
  KEY `emergency_id` (`emergency_id`),
  KEY `idx_admissions_patient_status` (`patient_id`,`status`),
  KEY `idx_admissions_bed_status` (`bed_id`,`status`),
  KEY `idx_admissions_doctor_status` (`doctor_id`,`status`),
  CONSTRAINT `admissions_ibfk_1` FOREIGN KEY (`patient_id`) REFERENCES `patients` (`patient_id`) ON DELETE RESTRICT,
  CONSTRAINT `admissions_ibfk_2` FOREIGN KEY (`doctor_id`) REFERENCES `doctors` (`doctor_id`) ON DELETE RESTRICT,
  CONSTRAINT `admissions_ibfk_3` FOREIGN KEY (`bed_id`) REFERENCES `beds` (`bed_id`) ON DELETE RESTRICT,
  CONSTRAINT `admissions_ibfk_4` FOREIGN KEY (`department_id`) REFERENCES `departments` (`department_id`) ON DELETE RESTRICT,
  CONSTRAINT `admissions_ibfk_5` FOREIGN KEY (`emergency_id`) REFERENCES `emergency_cases` (`emergency_id`) ON DELETE SET NULL,
  CONSTRAINT `admissions_chk_1` CHECK (((`discharge_date` is null) or (`discharge_date` >= `admission_date`)))
) ENGINE=InnoDB;
```

Listing 8.5: `prescription_items`: structured dosing and the dispensed CHECK

```
CREATE TABLE `prescription_items` (
  `item_id` int NOT NULL AUTO_INCREMENT,
  `prescription_id` int NOT NULL,
  `medicine_id` int NOT NULL,
  `dosage` varchar(50) NOT NULL,
  `frequency` varchar(50) NOT NULL,
  `duration` varchar(50) NOT NULL,
  `quantity` int NOT NULL,
  `dispensed` tinyint(1) DEFAULT '0',
  `dispensed_at` datetime DEFAULT NULL,
  `frequency_code` enum('OD','BD','TDS','QID','Q6H','Q8H','STAT','PRN') DEFAULT NULL,
  `duration_days` smallint DEFAULT NULL,
  `route` enum('ORAL','IV','IM','SC','SUBLINGUAL','INHALED','TOPICAL','RECTAL','OTHER') DEFAULT NULL,
  `units_per_dose` smallint DEFAULT NULL,
  PRIMARY KEY (`item_id`),
  KEY `prescription_id` (`prescription_id`),
  KEY `medicine_id` (`medicine_id`),
  CONSTRAINT `prescription_items_ibfk_1` FOREIGN KEY (`prescription_id`) REFERENCES `prescriptions` (`prescription_id`) ON DELETE
      CASCADE,
  CONSTRAINT `prescription_items_ibfk_2` FOREIGN KEY (`medicine_id`) REFERENCES `medicines` (`medicine_id`) ON DELETE RESTRICT,
  CONSTRAINT `chk_prescription_items_dispensed` CHECK (((`dispensed_at` is null) or (`dispensed` = 1))),
  CONSTRAINT `chk_prescription_items_structured` CHECK ((((`duration_days` is null) or (`duration_days` > 0)) and ((`
      units_per_dose` is null) or (`units_per_dose` > 0)))),
  CONSTRAINT `prescription_items_chk_1` CHECK ((`quantity` > 0))
) ENGINE=InnoDB;
```

Listing 8.6: `medicine_batches`: a weak entity (UNIQUE owner + batch number)

```
CREATE TABLE `medicine_batches` (
  `batch_id` int NOT NULL AUTO_INCREMENT,
  `medicine_id` int NOT NULL,
  `batch_number` varchar(50) NOT NULL,
  `quantity` int NOT NULL,
  `expiry_date` date NOT NULL,
  `received_date` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `received_by` int DEFAULT NULL,
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`batch_id`),
  UNIQUE KEY `uq_medicine_batches_number` (`medicine_id`,`batch_number`),
```

```
  UNIQUE KEY `uq_medicine_batches_id_medicine` (`batch_id`,`medicine_id`),
  KEY `idx_medicine_batches_fefo` (`medicine_id`,`expiry_date`,`batch_id`),
  KEY `fk_medicine_batches_received_by` (`received_by`),
  CONSTRAINT `fk_medicine_batches_medicine` FOREIGN KEY (`medicine_id`) REFERENCES `medicines` (`medicine_id`) ON DELETE RESTRICT,

  CONSTRAINT `fk_medicine_batches_received_by` FOREIGN KEY (`received_by`) REFERENCES `users` (`user_id`) ON DELETE RESTRICT,
  CONSTRAINT `chk_medicine_batches_quantity` CHECK ((`quantity` >= 0))
) ENGINE=InnoDB;
```

Listing 8.7: `bill_items`: the generated total and CASCADE from its bill

```
CREATE TABLE `bill_items` (
  `item_id` int NOT NULL AUTO_INCREMENT,
  `bill_id` int NOT NULL,
  `description` varchar(255) NOT NULL,
  `category` enum('CONSULTATION','BED','ICU','LABORATORY','MEDICINE','PROCEDURE','AMBULANCE','OTHER') NOT NULL,
  `quantity` int DEFAULT '1',
  `unit_price` decimal(10,2) NOT NULL,
  `total_price` decimal(10,2) GENERATED ALWAYS AS ((`quantity` * `unit_price`)) VIRTUAL NOT NULL,
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`item_id`),
  KEY `bill_id` (`bill_id`),
  CONSTRAINT `bill_items_ibfk_1` FOREIGN KEY (`bill_id`) REFERENCES `bills` (`bill_id`) ON DELETE CASCADE,
  CONSTRAINT `bill_items_chk_1` CHECK ((`quantity` > 0)),
  CONSTRAINT `bill_items_chk_2` CHECK ((`unit_price` >= 0)),
  CONSTRAINT `bill_items_chk_3` CHECK ((`total_price` >= 0))
) ENGINE=InnoDB;
```

8.3 Data Insertion

Listing 8.8: Inserting a patient and a bill line (results below)

```
INSERT INTO patients (name, date_of_birth, gender, blood_group, phone, abha_number)
VALUES ('Meera Iyer', '1992-07-14', 'FEMALE', 'B+', '9810012345', '12345678901234');
INSERT INTO bill_items (bill_id, description, category, quantity, unit_price)
VALUES (1, 'Dressing change', 'PROCEDURE', 3, 150.00);
```

Result: 1 row each. The patient got `patient_id = 37`. The bill line got `total_price = 450.00` without the statement supplying it (generated column). The seed data (`database/seed.sql`) inserts 17 staff accounts, 12 doctors, 9 departments, 6 wards, 35 beds, 32 patients, 22 medicines, 15 lab tests and their activity.

8.4 Data Modification and Deletion

Listing 8.9: UPDATE and DELETE, including a cascade and a refused delete (results below)

```
UPDATE bill_items SET quantity = 4 WHERE item_id = 20;
UPDATE beds SET status = 'MAINTENANCE' WHERE bed_id = 1 AND status = 'AVAILABLE';
DELETE FROM bills WHERE bill_id = 1; -- its bill_items go with it (ON DELETE CASCADE)
DELETE FROM patients WHERE patient_id = 1; -- refused: the patient is referenced (RESTRICT)
```

Results: the bill line's total became 600.00 (4×150.00); bed GEN-A01 became MAINTENANCE (1 row); deleting bill 1 removed 1 bill and its 8 items ($8 \rightarrow 0$); deleting patient 1 failed with `ER_ROW_IS_REFERENCED_2 (1451)`: foreign key `bed_assignment_log_ibfk_2` (`ON DELETE RESTRICT`).

8.5 Sample Database Contents

Table 8.1: Sample contents of `patients`, `departments` and beds per ward (`SELECT` on the test database)

patient_id	name	date_of_birth	gender	blood_group
1	Aarav Sharma	1985-03-15	MALE	O+
2	Diya Patel	1990-07-22	FEMALE	A+
3	Vihaan Reddy	1978-11-30	MALE	B+
4	Ananya Gupta	1995-05-10	FEMALE	AB+
5	Aditya Kumar	1960-01-20	MALE	O-

department_id	name	floor
1	Emergency	1
2	Cardiology	2
3	Neurology	3
4	Orthopedics	2
5	General Medicine	1
6	Pediatrics	1

ward	beds	available
General Ward A	10	8
General Ward B	8	7
Cardiac Ward	4	2
Pediatric Ward	3	2
ICU Ward	6	2
Emergency Observation	4	3

8.6 SQL Query Implementation

The fifteen queries below cover the ten required kinds, and most are the queries the application itself runs or reports on. Each gives its SQL, purpose and real output.

Listing 8.10: Query 1 (Simple SELECT with conditions). Medicines whose stock has fallen to or below their reorder level (the low-stock alert).

```
SELECT medicine_id, name, stock_quantity, reorder_level
FROM medicines
WHERE stock_quantity <= reorder_level AND requires_prescription
     = 1;
```

Table 8.2: Output of Query 1

medicine_id	name	stock_quantity	reorder_level
21	Morphine Sulphate 10mg	50	50

1 row

Listing 8.11: Query 2 (ORDER BY). Available ICU and ventilator beds, ordered by ward and bed number (what the emergency desk looks at first).

```
SELECT b.bed_number, b.bed_type, w.name AS ward, w.floor
FROM beds b JOIN wards w ON w.ward_id = b.ward_id
WHERE b.status = 'AVAILABLE' AND b.bed_type IN ('ICU', '
     VENTILATOR_ICU')
ORDER BY w.name, b.bed_number;
```

Table 8.3: Output of Query 2

bed_number	bed_type	ward	floor
ICU-04	ICU	ICU Ward	3
VICU-02	VENTILATOR_ICU	ICU Ward	3

2 rows

Listing 8.12: Query 3 (Aggregates: COUNT, SUM, AVG, MIN, MAX). Billing summary: how many bills, how much billed and paid, and the smallest, average and largest bill.

```
SELECT COUNT(*) AS bills, SUM(total_amount) AS billed, SUM(
     paid_amount) AS paid,
      MIN(total_amount) AS smallest, ROUND(AVG(total_amount),
           2) AS average,
      MAX(total_amount) AS largest
FROM bills;
```

Table 8.4: Output of Query 3

bills	billed	paid	smallest	average	largest
4	246500.00	86500.00	1500.00	61625.00	125000.00

1 row

Listing 8.13: Query 4 (GROUP BY with HAVING). Departments that have more than one doctor, with their doctor count and combined capacity.

```
SELECT dep.name AS department, COUNT(*) AS doctors,
      SUM(doc.max_workload) AS capacity
FROM doctors doc JOIN departments dep ON dep.department_id = doc
     .department_id
GROUP BY dep.department_id, dep.name
HAVING COUNT(*) > 1
ORDER BY doctors DESC, department;
```

Table 8.5: Output of Query 4

department	doctors	capacity
Cardiology	2	10
Emergency	2	16
General Medicine	2	10
ICU	2	8
Neurology	2	10

5 rows

Listing 8.14: Query 5 (INNER JOIN). Today's appointments with patient and doctor names; the department comes from the doctor through the view v_appointments.

```
SELECT a.appointment_time, p.name AS patient, d.name AS doctor,
      dep.name AS department, a.status
FROM v_appointments a
INNER JOIN patients p ON p.patient_id = a.patient_id
INNER JOIN doctors d ON d.doctor_id = a.doctor_id
INNER JOIN departments dep ON dep.department_id = a.
     department_id
WHERE a.appointment_date = CURDATE()
ORDER BY a.appointment_time;
```

Table 8.6: Output of Query 5

appointment_time	patient	doctor	department	status
09:00:00	Aarav Sharma	Dr. John Smith	Cardiology	BOOKED
09:30:00	Rohan Mehta	Dr. Lakshmi Reddy	Pediatrics	COMPLETED
10:00:00	Diya Patel	Dr. Vikram Sharma	Orthopedics	BOOKED
10:30:00	Ananya Gupta	Dr. Priya Patel	Neurology	CHECKED_IN
11:00:00	Kavya Joshi	Dr. John Smith	Cardiology	BOOKED

9 rows; first 5 shown

Listing 8.15: Query 6 (LEFT JOIN). Every doctor with the number of ACTIVE admissions they hold, including doctors with none (LEFT JOIN keeps them, COUNT gives 0).

```
SELECT d.name, d.current_workload, d.max_workload,
      COUNT(a.admission_id) AS active_admissions
FROM doctors d
LEFT JOIN admissions a ON a.doctor_id = d.doctor_id AND a.status
      = 'ACTIVE'
GROUP BY d.doctor_id, d.name, d.current_workload, d.max_workload
ORDER BY active_admissions DESC, d.name
LIMIT 6;
```

Table 8.7: Output of Query 6

name	current_workload	max_workload	active_admissions
Dr. Neha Mehta	3	5	3
Dr. John Smith	2	5	2
Dr. Arjun Singh	1	4	1
Dr. Lakshmi Reddy	1	6	1
Dr. Rajesh Kumar	1	8	1

6 rows; first 5 shown

Listing 8.16: Query 7 (Self join (another join type)). Pairs of doctors who share a specialization: the doctors table joined to itself.

```
SELECT d1.specialization, d1.name AS doctor, d2.name AS
     colleague
FROM doctors d1
JOIN doctors d2 ON d2.specialization = d1.specialization
             AND d2.doctor_id > d1.doctor_id
ORDER BY d1.specialization;
```

Table 8.8: Output of Query 7

specialization	doctor	colleague
Cardiology	Dr. John Smith	Dr. Neha Mehta
Critical Care	Dr. Arjun Singh	Dr. Ritu Agarwal
Emergency Medicine	Dr. Rajesh Kumar	Dr. Sunil Joshi
General Medicine	Dr. Anita Gupta	Dr. Amit Das
Neurology	Dr. Priya Patel	Dr. Kavitha Rao

5 rows

Listing 8.17: Query 8 (Nested query (subquery in WHERE)). Medicines priced above the average price of all medicines.

```
SELECT name, category, unit_price
FROM medicines
WHERE unit_price > (SELECT AVG(unit_price) FROM medicines)
ORDER BY unit_price DESC
LIMIT 6;
```

Table 8.9: Output of Query 8

name	category	unit_price
Insulin Glargine	Insulin	850.00
Heparin 5000IU	Anticoagulant	180.00
Salbutamol Inhaler	Bronchodilator	120.00

3 rows

Listing 8.18: Query 9 (Correlated subquery). Each admitted patient with the time of their latest vital-signs reading (NULL: not taken yet). The subquery runs once per admission.

```
SELECT p.name, a.admission_date,
       (SELECT MAX(v.recorded_at_utc) FROM vital_signs v
         WHERE v.admission_id = a.admission_id) AS
              last_vitals_utc
FROM admissions a JOIN patients p ON p.patient_id = a.patient_id
WHERE a.status = 'ACTIVE'
ORDER BY a.admission_date DESC
LIMIT 6;
```

Table 8.10: Output of Query 9

name	admission_date	last_vitals_utc
Shruti Menon	2026-10-13 13:53	NULL
Arjun Nair	2026-10-12 19:53	NULL
Rajendra Mishra	2026-10-12 19:53	NULL
Deepak Choudhary	2026-10-12 19:53	NULL
Vihaan Reddy	2026-10-11 19:53	NULL

6 rows; first 5 shown

Listing 8.19: Query 10 (IN and EXISTS). Patients with an ACTIVE admission (EXISTS) who also have a lab order that is not yet completed (IN).

```
SELECT p.patient_id, p.name
FROM patients p
WHERE EXISTS (SELECT 1 FROM admissions a
              WHERE a.patient_id = p.patient_id AND a.status = '
                   ACTIVE')
  AND p.patient_id IN (SELECT o.patient_id FROM lab_orders o
                       WHERE o.status <> 'COMPLETED');
```

Table 8.11: Output of Query 10

patient_id	name
7	Arjun Nair

1 row

Listing 8.20: Query 11 (ALL and ANY). Medicines dearer than every antibiotic (> ALL), and antacids dearer than at least one antiplatelet (> ANY).

```
SELECT name, category, unit_price FROM medicines
WHERE unit_price > ALL (SELECT unit_price FROM medicines WHERE
     category = 'Antibiotic')
ORDER BY unit_price DESC;
SELECT name, unit_price FROM medicines
WHERE category = 'Antacid'
  AND unit_price > ANY (SELECT unit_price FROM medicines WHERE
       category = 'Antiplatelet');
```

Table 8.12: Output of Query 11

name	category	unit_price
Insulin Glargine	Insulin	850.00
Heparin 5000IU	Anticoagulant	180.00
Salbutamol Inhaler	Bronchodilator	120.00
Diclofenac Gel	Topical NSAID	65.00
Morphine Sulphate 10mg	Opioid Analgesic	45.00

5 rows

name	unit_price
Omeprazole 20mg	12.00
Pantoprazole 40mg	14.00

2 rows

Listing 8.21: Query 12 (Multiple tables (five-table join)). Current inpatients with ward, bed, admitting doctor and department.

```
SELECT p.name AS patient, w.name AS ward, b.bed_number,
       d.name AS doctor, dep.name AS department
FROM admissions a
JOIN patients p ON p.patient_id = a.patient_id
JOIN beds b ON b.bed_id = a.bed_id
JOIN wards w ON w.ward_id = b.ward_id
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN departments dep ON dep.department_id = a.department_id
WHERE a.status = 'ACTIVE'
ORDER BY w.name, b.bed_number
LIMIT 6;
```

Table 8.13: Output of Query 12

patient	ward	bed_number	doctor	department
Rajendra Mishra	Cardiac Ward	CARD-02	Dr. John Smith	Cardiology
Sneha Rao	Cardiac Ward	CARD-04	Dr. Lakshmi Reddy	General Medicine
Deepak Choudhary	Emergency Observation	ISO-02	Dr. Rajesh Kumar	Emergency
Vihaan Reddy	General Ward A	GEN-A03	Dr. John Smith	Cardiology
Suresh Pillai	General Ward B	GEN-B04	Dr. Vikram Sharma	General Medicine

6 rows; first 5 shown

Listing 8.22: Query 13 (Business question: bed occupancy). Occupancy by bed type: how full is the hospital? (CASE inside SUM.)

```
SELECT bed_type, COUNT(*) AS beds,
      SUM(status = 'OCCUPIED') AS occupied,
      ROUND(100 * SUM(status = 'OCCUPIED') / COUNT(*), 1) AS
           occupancy_pct
FROM beds
GROUP BY bed_type
ORDER BY occupancy_pct DESC;
```

Table 8.14: Output of Query 13

bed_type	beds	occupied	occupancy_pct
ICU	4	2	50.0
VENTILATOR_ICU	2	1	50.0
NICU	3	1	33.3
ISOLATION	4	1	25.0
GENERAL	22	4	18.2

5 rows

Listing 8.23: Query 14 (Business question: stock that expires soonest). For each medicine with stock, its usable quantity and next expiry, from the batches (view v_current_stock).

```
SELECT name, stock_quantity, usable_quantity, next_expiry
FROM v_current_stock
WHERE usable_quantity > 0
ORDER BY next_expiry, name
LIMIT 6;
```

Table 8.15: Output of Query 14

name	stock_quantity	usable_quantity	next_expiry
Amlodipine 5mg	400	400	2027-03-10
Morphine Sulphate 10mg	50	50	2027-04-20
Pantoprazole 40mg	450	450	2027-05-15
Insulin Glargine	100	100	2027-06-15
Amoxicillin 250mg	500	500	2027-06-30

6 rows; first 5 shown

Listing 8.24: Query 15 (View). The view current_admissions_view joins admissions with patients, doctors, beds, wards and departments; callers select from it like a table.

```
SELECT patient_name, ward_name, bed_number, doctor_name,
     department_name
FROM current_admissions_view
ORDER BY ward_name, bed_number
LIMIT 6;
```

Table 8.16: Output of Query 15

patient_name	ward_name	bed_number	doctor_name	department_name
Rajendra Mishra	Cardiac Ward	CARD-02	Dr. John Smith	Cardiology
Sneha Rao	Cardiac Ward	CARD-04	Dr. Lakshmi Reddy	General Medicine
Deepak Choudhary	Emergency Observation	ISO-02	Dr. Rajesh Kumar	Emergency
Vihaan Reddy	General Ward A	GEN-A03	Dr. John Smith	Cardiology
Suresh Pillai	General Ward B	GEN-B04	Dr. Vikram Sharma	General Medicine

6 rows; first 5 shown

8.7 Advanced DBMS Features

Views (10). `v_appointments` returns appointments with the department taken from the doctor (Section 6.3), so reads keep their columns after the decomposition; `v_current_stock` shows each medicine's total, usable and expired quantities and next expiry from its batches (Query 14); `current_admissions_view` (Query 15), `available_beds_view`, `bed_status_summary_view`, `hospital_occupancy_view`, `available_doctors_view`, `doctor_workload_view`, `emergency_queue_view` and `pending_bills_view` give reporting summaries.

Triggers (7). `after_admission_insert` (Listing 8.25) and `after_admission_discharge` keep beds, doctor workload and the bed history in step with admissions; `after_prescription_dispense` takes a dispensed quantity off `medicines.stock_quantity`; `before_medicine_stock_update` refuses a negative total; `audit_logs_no_update` and `audit_logs_no_delete` make the audit log append-only; `after_bed_assignment_log` is an empty placeholder from the original schema.

Listing 8.25: Trigger `after_admission_insert` (since migration 020, workload counts only ACTIVE admissions)

```
CREATE TRIGGER after_admission_insert AFTER INSERT ON admissions FOR EACH ROW
BEGIN
    UPDATE beds SET status = 'OCCUPIED' WHERE bed_id = NEW.bed_id;
    -- Workload is the number of ACTIVE admissions; one inserted already discharged does not count.
    IF NEW.status = 'ACTIVE' THEN
        UPDATE doctors SET current_workload = current_workload + 1 WHERE doctor_id = NEW.doctor_id;
    END IF;

    INSERT INTO bed_assignment_log (bed_id, patient_id, admission_id, assigned_at, status)
    VALUES (NEW.bed_id, NEW.patient_id, NEW.admission_id, NEW.admission_date, 'ACTIVE');
END
```

Stored procedures (5), from the original schema: `sp_admit_patient`, `sp_discharge_patient`, `sp_allocate_emergency`, `sp_generate_bill` and `sp_get_emergency_queue`.

Transactions and locking. Every change runs in a transaction (`withTransaction` in `server/utils/audit.js`): the change and its audit row commit together or not at all. InnoDB runs at REPEATABLE READ, and operations that read a value and then change it lock the rows first (`SELECT ... FOR UPDATE`). To prevent deadlocks, rows are always locked in the same order. Dispensing, for example, locks the prescription, then its medicines in ascending id, then each medicine's batches by expiry date:

Listing 8.26: Lock order in one dispensing transaction (`dispenseService`)

```
START TRANSACTION;
SELECT status, patient_id FROM prescriptions WHERE prescription_id = ? FOR UPDATE;
SELECT medicine_id, stock_quantity FROM medicines WHERE medicine_id = ? FOR UPDATE; -- each, ascending id
```

```
SELECT batch_id, quantity, expiry_date FROM medicine_batches
WHERE medicine_id = ? AND quantity > 0 AND expiry_date >= CURDATE()
ORDER BY expiry_date, batch_id FOR UPDATE; -- first expiry first out
UPDATE prescription_items SET dispensed = TRUE, dispensed_at = NOW()
WHERE item_id IN (?) AND dispensed = FALSE; -- trigger after_prescription_dispense lowers the total
UPDATE medicine_batches SET quantity = quantity - ? WHERE batch_id = ?; -- each batch taken
INSERT INTO prescription_item_batches (item_id, batch_id, quantity) VALUES (?, ?, ?);
INSERT INTO pharmacy_stock_movements (medicine_id, movement_type, quantity, reason, reference_id, performed_by, batch_id)
VALUES (?, 'DISPENSE', ?, 'Dispense Prescription', ?, ?, ?);
UPDATE prescriptions SET status = 'DISPENSED' WHERE prescription_id = ?;
COMMIT;
```

The stress test runs 300 rounds of competing prescribing and dispensing on the same two medicines in opposite orders, and 300 rounds of mixed writers: 0 deadlocks, and every medicine's total equals the sum of its batches. Bed allocation locks the chosen bed and then the doctor (`FOR UPDATE`), so two clerks cannot give the same last ICU bed to two patients.

Chapter 9

Application Development

9.1 System Architecture

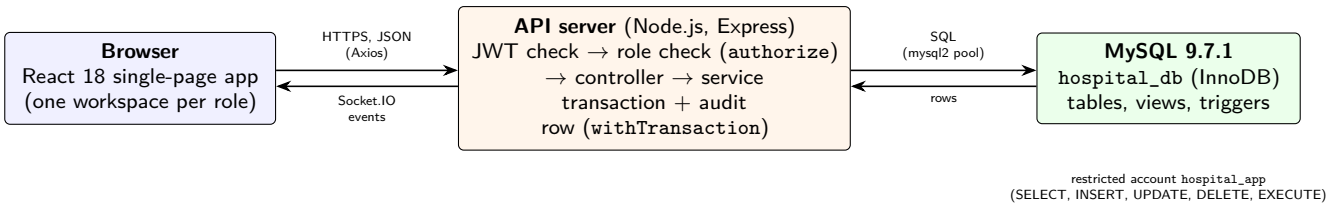


Figure 9.1: Architecture: three tiers; the database is reached only through the API's restricted account

9.2 Front-End Technologies

JavaScript throughout. The client is React 18 with React Router 6 (role-guarded routes), built with Vite 4, calling the API through Axios and receiving live updates (bed changes, the doctor's queue, the emergency queue) over the Socket.IO client. The API is Express 4 on Node.js with `mysql2/promise`, `jsonwebtoken` (sign-in tokens), `bcryptjs` (password hashes) and Multer (lab report uploads). Tests use `node:test` and Puppeteer.

9.3 Application Modules

The API has 24 routers under `/api`. They map to the ER modules:

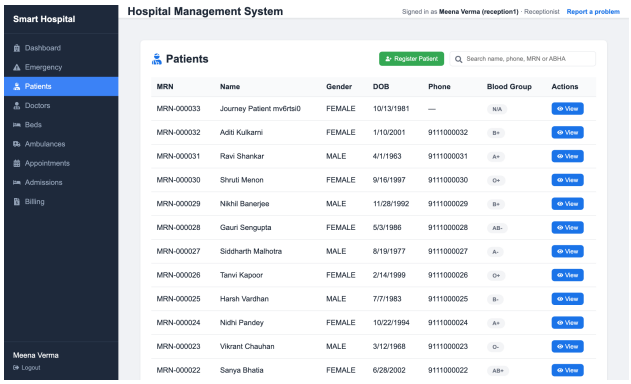
- **Patients and visits:** registration and search, appointments, the visit state machine (arrive, triage, start, finish), vital signs, consultation notes.
- **Clinical orders:** prescriptions with structured dosing, surgery requests; the nurse's MAR (doses due, given, refused, missed).
- **Pharmacy:** the dispense queue, dispensing by batch, stock receipt and adjustment, low-stock and expiry views.
- **Laboratory:** the order board, results with automatic flagging, report uploads and secure downloads.
- **Beds and admissions:** the bed board, admission and discharge, emergency queue and allocation, ambulances, billing.
- **Staff and audit:** staff accounts, nurse ward assignments, doctor dashboard statistics, the audit log viewer.

9.4 Database Connectivity

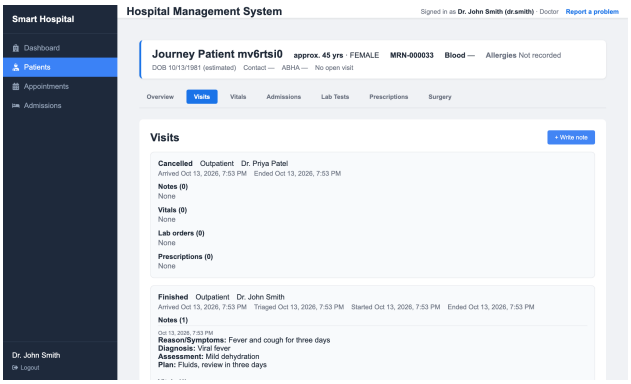
- **Pool:** `server/config/db.js` creates a `mysql2` connection pool (20 connections) from `server/.env`. All SQL uses placeholders (`?`), so user input is never spliced into a statement.
- **Restricted account:** the running API connects as `hospital_app`, which can read and write rows and call procedures but cannot change the schema; migrations use a separate administrative account.
- **Transactions:** `withTransaction(req, fn)` takes a connection, begins a transaction, runs the change and its audit row on that connection, then commits (or rolls back on any error) and releases it.
- **Access control:** every route checks the token and the role (`authorize`); patient data is further limited to a doctor's care set or a nurse's assigned wards (`utils/patientAccess.js`).

9.5 Application Screenshots

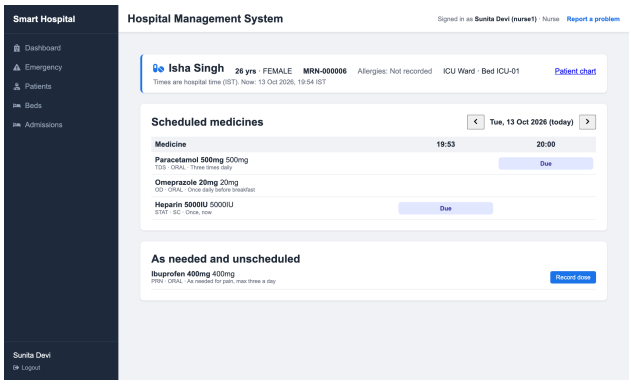
The screenshots (Figure 9.2 and Appendix C) were taken from the application running on the test database, which holds only made-up people.



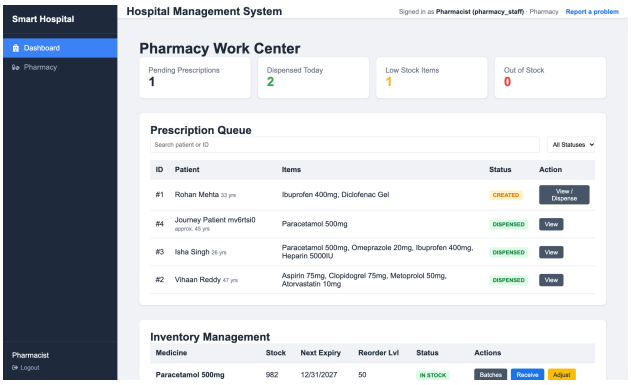
(a) Receptionist: patient list and search



(b) Doctor: a patient's visits, notes and orders



(c) Nurse: medication administration record



(d) Pharmacy: dispense queue and stock

Figure 9.2: Major screens, one per role (test data)

9.6 Database Operations through the Application

Each screen action becomes SQL in one transaction, and the result is visible in MySQL. The patient-journey browser test drives these screens and then checks the rows in the database.

Action on screen	Statements on hospital_db
Register a patient (reception)	`INSERT INTO patients`; `INSERT INTO audit_logs` (CREATE_PATIENT)
Book an appointment	`INSERT INTO appointments` (a duplicate slot fails the UNIQUE key: 409 "Double booking detected")
Mark arrived, start, finish a visit	`SELECT ... FOR UPDATE` on the appointment and encounter; `INSERT`/`UPDATE encounters`; `UPDATE appointments SET status`
Record vital signs (nurse)	`INSERT INTO vital_signs` (refused by the CHECKs if out of range)
Dispense (pharmacy)	Listing in Section 8.7: locks, `UPDATE` items and batches, `INSERT` movements
Admit, discharge (doctor)	`INSERT INTO admissions` / `UPDATE admissions SET status = 'DISCHARGED'`; triggers update the bed, the workload and `bed_assignment_log`
Edit a patient's details	`UPDATE patients`; the audit row names the changed fields, never their values
Cancel a prescription	`UPDATE prescriptions SET status = 'CANCELLED'`; pending doses `UPDATE medication_administrations SET status = 'CANCELLED'`

Rows are not deleted through the application: clinical records are kept, and corrections are recorded as new states (for example a visit marked ENTERED_IN_ERROR), which the audit log requires.

Chapter 10

Testing and Results

10.1 Test Cases

Each case was run against the API and database of the test environment; the actual result is the response or MySQL error that came back.

Table 10.1: Test cases with their actual results

Test ID	Operation	Input	Expected Result	Actual Result	Status
T1	Register a patient	Valid name, DOB 1990-04-12, gender	Patient created	HTTP 201, patient 34 created	Pass
T2	Duplicate key	users.username = 'admin' (exists)	Rejected by UNIQUE	ER_DUP_ENTRY (1062): Duplicate entry 'admin' for key 'users.username'	Pass
T3	Invalid foreign key	lab_orders.test_id = 9999 (no such test)	Rejected by FOREIGN KEY	ER_NO_REFERENCED_ROW_2 (1452): FK fails (constraint lab_orders_ibfk_4)	Pass
T4	Negative stock	medicines.stock_quantity = -5	Rejected (trigger and CHECK)	ER_SIGNAL_EXCEPTION (1644): Error: Medicine stock cannot be negative.	Pass
T5	Future date of birth	date_of_birth = 2026-11-12	Rejected with 400	HTTP 400: Date of birth cannot be in the future	Pass
T6	Duplicate ABHA number	Second patient with the same 14-digit ABHA	Rejected with 409	HTTP 201 then 409: ABHA number 91-0000-0062-3238 is already registered to another patient	Pass
T7	Double dispense	Dispense prescription 2 again (already DISPENSED)	Rejected, stock unchanged	HTTP 409: Prescription is not available for dispensing	Pass
T8	Cross-role access	Receptionist reads a patient's clinical history	Refused with 403	HTTP 403: Forbidden: this is available to doctors, nurses only	Pass
T9	Wrong role for an action	Doctor tries to dispense a prescription	Refused with 403	HTTP 403: Forbidden: this is available to pharmacy staff only	Pass
T10	Copy disagrees with its link	Bill for patient 34 on another patient's admission 1	Rejected with 400	HTTP 400: That admission belongs to a different patient	Pass
T11	CHECK constraint	Dispense time on an item not marked dispensed	Rejected by CHECK	ER_CHECK_CONSTRAINT_VIOLATED (3819): Check constraint 'chk_prescription_items_dispensed' is violated.	Pass
T12	Tamper with the audit log	UPDATE audit_logs	Rejected by trigger	ER_SIGNAL_EXCEPTION (1644): audit_logs is append-only: rows cannot be updated	Pass
T13	Double booking	Same doctor, date and time twice	Second refused with 409	HTTP 201 then 409: Double booking detected	Pass

10.2 Query Testing

Each query of Section 8.6 was run on the test database and its output checked against the data. For example, the bed totals agree (35 beds both by type in Query 13 and by ward in Table 8.1), Query 6's LEFT JOIN returns doctors with no active admission as 0 rather than dropping them, and Query 11's `> ALL` returns exactly the medicines dearer than the most expensive antibiotic (25.00). The automated suite also tests the application's own queries: for instance, that a doctor's patient list contains only patients in their care, and that every medicine's total equals the sum of its batches after each stock test.

10.3 Constraint Testing

Cases T2 to T4, T6, T11 and T12 show the database refusing invalid data on its own (duplicate key, missing parent, negative stock, duplicate ABHA, a dispense time without the flag, editing the audit log). T10 shows a composite foreign key and the matching API check refusing a bill whose patient differs from its admission's patient. The server suite contains a test for every constraint added in this work, run on the database itself.

10.4 Results and Discussion

- **Server suite:** 282 tests in 42 files (`node:test`) against a server on `hospital_db_test`, which is rebuilt from the migrations for every run: all pass.
- **Browser suite:** 121 tests in 29 files (Puppeteer) driving every role's screens: all pass.
- **Stress test:** 300 rounds of competing prescribing and dispensing, and 300 rounds of mixed writers: 0 deadlocks, stock consistent.
- **Patient journey:** 13 steps from registration to discharge across five roles (every role but laboratory): all pass, locally and against the deployed site.

One defect was found while preparing this report: `POST /api/patients` answers HTTP 500 instead of saving when the request omits the optional fields (blood group, phone, address, emergency contact), because the

missing values reach MySQL as `undefined`. The registration form always sends them, so users are not affected; it is listed as a limitation and was not changed for this report.

Chapter 11

Conclusion and Future Scope

11.1 Conclusion

The project models a hospital's operations as 30 entities and 69 relationships, maps them to 33 MySQL tables, and normalizes them: 24 tables are in BCNF, and the 9 that keep a copied value have that copy enforced by composite foreign keys or a CHECK constraint. The rules of the hospital live in the database (87 foreign keys, 23 unique keys, 30 CHECKs, 7 triggers, row locking in a fixed order), and a role-based React and Express application uses it through a restricted account. Automated tests confirm the behaviour, including under concurrency.

11.2 Limitations

- A lab order stores one result value: multi-analyte panels (a lipid profile's four values) are recorded as text in one result.
- The ABHA number is stored and validated for format and uniqueness only; there is no live integration with the national ABDM network.
- Allergies are free text in one column, not a separate table, so they cannot be checked against a prescription automatically.
- Bed transfers within one admission are not modelled; an admission has one bed.
- `POST /api/patients` returns HTTP 500 when optional fields are omitted from the request (Section 10.4).
- Seven of the ten views and the five stored procedures come from the original schema and are not used by the API.
- One hospital only; no payment gateway or insurance claims.

11.3 Future Scope

- Panel results (one row per analyte) and a structured allergy list checked at prescribing time.
- ABDM integration: verify ABHA numbers and exchange health records with consent.
- Bed transfers, with the bed history as the source of truth for where a patient is.
- Payments and insurance claims; dashboards over the audit log and the daily statistics.

References

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[3] P. P.-S. Chen, "The entity-relationship model: toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[4] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[5] Oracle, *MySQL 9.7 Reference Manual*, chapters on InnoDB locking, foreign keys, CHECK constraints, generated columns and triggers. [Online]. Available: `https://dev.mysql.com/doc/`

[6] OpenJS Foundation, *Node.js Documentation* (including the `node:test` runner). [Online]. Available: `https://nodejs.org/docs/`

[7] *Express 4 API Reference*. [Online]. Available: `https://expressjs.com/`

[8] Meta, *React Documentation*. [Online]. Available: `https://react.dev/`

[9] *mysql2 for Node.js*. [Online]. Available: `https://sidorares.github.io/node-mysql2/docs`

[10] Google, *Puppeteer Documentation*. [Online]. Available: `https://pptr.dev/`

[11] National Health Authority, *Ayushman Bharat Digital Mission: ABHA number*. [Online]. Available: `https://abdm.gov.in/`

Appendix A

Complete MySQL Script

The complete script is supplied as a separate file with this report: **docs/report/hospital_db_full.sql** (about 130 KB). It was exported with `mysqldump` from a database built from `database/seed.sql` and migrations 000–023, the same schema as `hospital_db`, and contains, in order:

- `CREATE DATABASE hospital_db` (utf8mb4) and `USE hospital_db`;
- the 33 `CREATE TABLE` statements with every primary key, foreign key (including the 15 composite consistency keys), UNIQUE key, CHECK constraint, index and the generated column;
- the sample data as `INSERT` statements (all people in it are made up; account passwords are bcrypt hashes of the documented demo password);
- the 10 views, 7 triggers and 5 stored procedures.

Load it with `mysql -u root -p < hospital_db_full.sql`. Loaded into an empty server, it produced a database whose structure matched `hospital_db` exactly (schema-diff: no differences). The queries of Section 8.6 are in `docs/report/generated/q1.sql` to `q15.sql`, and the migrations that build the schema step by step are in `database/migrations/`.

Appendix B

Source Code

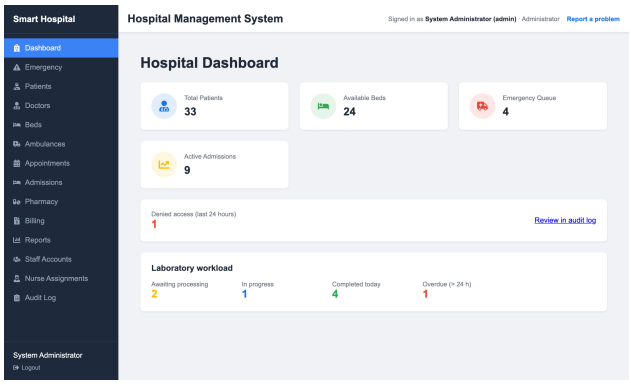
The source code is in the GitHub repository **https://github.com/rohithsaivemuri-sudo/hospital-dashboard** (branch `main`):

- `server/`: the Express API (routes, controllers, services, middleware), migration and test scripts, and the automated tests (`server/tests`);
- `client/`: the React application (`client/src/pages` has one folder per module);
- `database/`: the migrations and the seed data;
- `tests/ui/`: the browser tests; `deploy/azure/`: deployment scripts;
- `docs/`: this report and its source (`docs/report`), the BCNF analysis, the database report and the editable ER diagrams (`er-diagrams.drawio`).

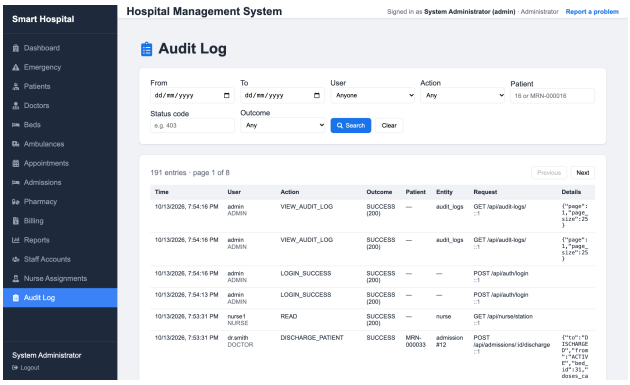
A demonstration copy runs at `https://hms-demo-in.indonesiacentral.cloudapp.azure.com` (sign-in required).

Appendix C

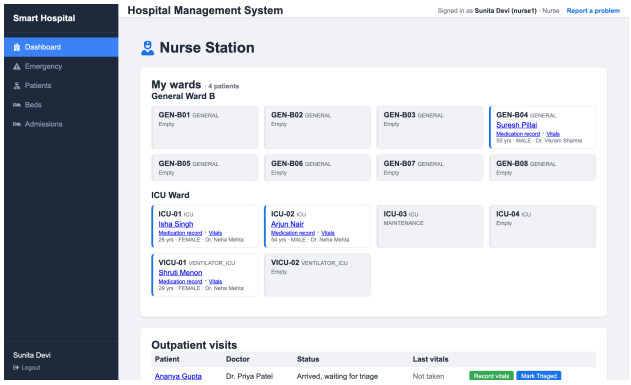
Additional Screenshots



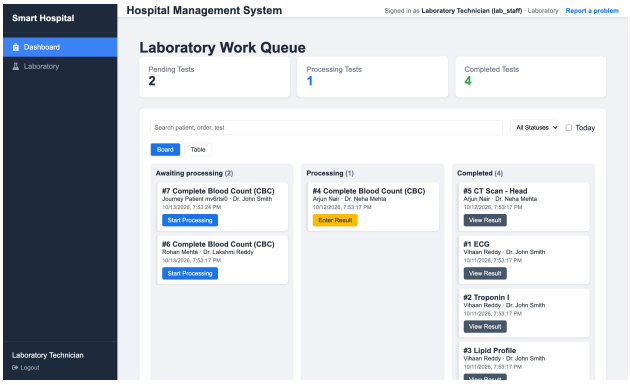
(a) Administrator: hospital dashboard



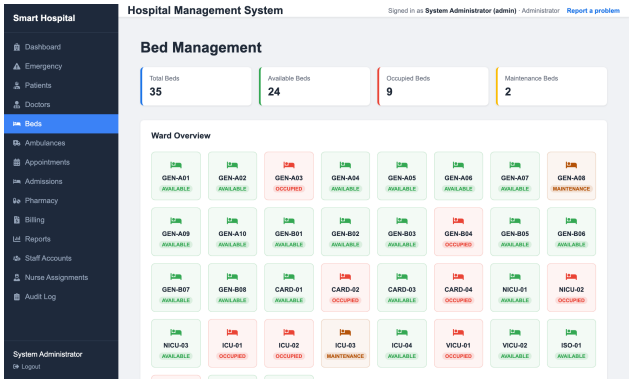
(b) Administrator: audit log



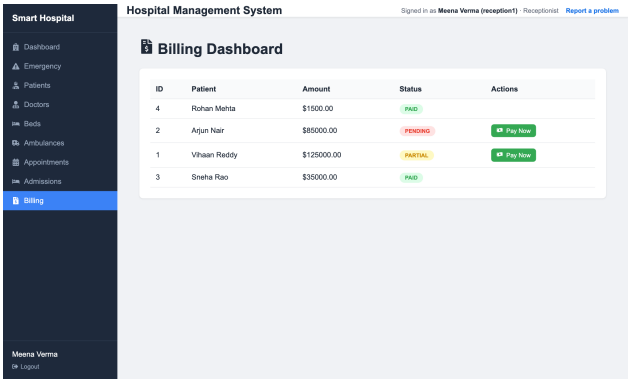
(c) Nurse: ward station with assigned beds



(d) Laboratory: order board



(e) Bed board



(f) Receptionist: billing

Figure C.1: Further screens (test data)

Appendix D

Team Contribution

Team member	Roll number	Contribution
[name]	*[roll no.]*	*[contribution]*
[name]	*[roll no.]*	*[contribution]*
[name]	*[roll no.]*	*[contribution]*
[name]	*[roll no.]*	*[contribution]*

Suggested Evaluation Scheme

Component	Marks	Where in this report
Problem analysis and requirements	5	Chapters 1, 2
ER diagram and constraints	10	Chapter 3
ER-to-relational mapping	10	Chapter 4
Functional dependencies and keys	10	Chapter 5
Normalization	15	Chapter 6
MySQL schema, constraints, and sample data	10	Chapters 7, 8.1–8.5, Appendix A
SQL queries and advanced DBMS features	15	Sections 8.6, 8.7
Front-end integration and functionality	10	Chapter 9
Testing and report quality	5	Chapter 10
Viva and conceptual understanding	10	–
Total	**100**	

Evaluation principle: Marks should primarily reflect the correctness of database design, ER-to-relational mapping, functional dependencies, normalization, SQL implementation, and understanding of DBMS concepts. Front-end appearance alone must not determine the project grade.